

Universidad Autónoma de Nuevo León

Facultad de ciencias Fisico Matematicas Minería de Datos

Producto Integral de Aprendizaje

Técnicas de análisis sobre el dataset de la Ultimate Fighting Championship (UFC)

Equipo #4

Nombre:

Brando Jahir Reyna Marroquin
Eder Abisai Esquivel Maldonado
Edgar Aurelio Santiago Santiago

Matricula:

1950966
2029995
1987398

Índice

Resumen-----	3
Introducción-----	3
Contexto-----	4
Caso de estudio-----	5
Descripción de técnicas-----	7
Tipo de análisis-----	8
Proceso de minería de datos-----	9
Resultados-----	37
Conclusiones-----	40
Referencias-----	42

Resumen

El presente documento corresponde al Producto Integral de Aprendizaje de la materia de Minería de Datos, en el cual se aplican técnicas de análisis sobre el dataset de la Ultimate Fighting Championship (UFC) que comprende registros de combates desde 1994 hasta 2026. El dataset se divide en dos partes: datos a nivel de peleador y datos a nivel de combate. A partir de esta información se definieron cuatro problemas analíticos que se abordan mediante técnicas de agrupamiento (K-Means Clustering), clasificación (Árboles de Decisión), regresión no lineal (Polinomial) y análisis descriptivo multicriterio (Ranking de Dominancia con normalización Min-Max). El objetivo general es extraer patrones ocultos en los datos de combate para segmentar estilos tácticos de pelea, identificar factores asociados a finalizaciones, modelar el efecto de la edad en el rendimiento atlético y construir un índice de dominancia objetivo libre de sesgos mediáticos. Los resultados obtenidos demuestran que la minería de datos permite transformar estadísticas deportivas en conocimiento estructurado y accionable.

Introducción

Actualmente los deportes de contacto son algo muy común y que se han convertido en un tema de interés para las personas debido a creadores de contenido que promocionan estos, por ende el análisis de los datos de los combates se convierte en una herramienta muy importante para comprender el rendimiento de los peleadores y cuáles son las fortalezas y debilidades de los mismos. El objetivo de este proyecto es aplicar los conocimientos aprendidos en clase con la finalidad de generar modelos capaces de analizar características de los datos de los peleadores y predecir ciertos sucesos.

Para alcanzar estos objetivos se recurrirá a técnicas de minería de datos debido a su capacidad para procesar grandes volúmenes de información y detectar patrones que no serían visibles mediante análisis estadísticos tradicionales. En el caso de la UFC, donde cada combate genera decenas de variables simultáneas, estas técnicas permiten ir más allá de los promedios básicos y descubrir relaciones complejas entre los datos.

Se esperan los siguientes beneficios como resultado del proyecto: dividir a los peleadores en diferentes categorías según el comportamiento táctico de sus combates anteriores, identificar las características estadísticas asociadas a finalizaciones por knockout o sumisión, modelar el deterioro físico relacionado con la edad del atleta y construir un ranking de dominancia basado en indicadores de eficiencia competitiva, libre de sesgos mediáticos o subjetivos.

Contexto

¿Qué es la minería de datos?

El data mining, o minería de datos, es una disciplina dentro del big data que permite analizar grandes volúmenes de información para identificar patrones, tendencias y correlaciones. El origen de esta ciencia se remonta a la década de 1960. Sin embargo, comenzó a adquirir mayor relevancia en los años 90 y 2000, gracias al avance de las tecnologías de procesamiento de datos. Esta tecnología se basa en un conjunto de técnicas y herramientas capaces de explorar bases de datos de forma automática o semiautomática, que transforman datos que parecen inconexos en información valiosa. Se le compara con una arqueología moderna, en la que los datos, de forma aislada, pueden parecer irrelevantes, pero al analizarlos en conjunto, revelan conocimientos estratégicos para la toma de decisiones.

Para lograr estos descubrimientos, la minería de datos se apoya en técnicas principales como la clasificación (asignar elementos a categorías predefinidas), la regresión (modelar relaciones para predecir valores continuos), el agrupamiento o clustering (descubrir segmentos naturales sin intervención humana) y las reglas de asociación (encontrar variables que ocurren juntas frecuentemente).

Ventajas y utilidad de la minería de datos en nuestro caso de estudio

En el área del deporte profesional y el entretenimiento, específicamente en la Ultimate Fighting Championship (UFC), utilizar la minería de datos otorga una ventaja analítica incalculable. Tradicionalmente, el análisis en las artes marciales mixtas dependía de la intuición, promedios básicos y la experiencia visual de entrenadores o periodistas.

Entre las principales ventajas de aplicar minería de datos en el área del entretenimiento deportivo se encuentran la capacidad de analizar simultáneamente cientos de variables por combate, la eliminación del sesgo humano en la evaluación del rendimiento, la posibilidad de detectar tendencias a lo largo de décadas de historia competitiva y la generación de predicciones basadas en evidencia estadística en lugar de intuición. Esto convierte a la minería de datos en una herramienta especialmente poderosa en deportes multifactoriales como las artes marciales mixtas, donde el resultado de un combate depende de una combinación compleja de factores físicos, técnicos y estratégicos.

Para nuestro caso de estudio, aplicar minería de datos resulta de extrema utilidad porque transforma un deporte multifactorial y aparentemente caótico en un

ecosistema medible. La interacción de cientos de variables procesadas por algoritmos nos permite ir más allá de la estadística básica, revelando qué factores determinan realmente el éxito en el octágono, prediciendo la caída del rendimiento atlético y construyendo evaluaciones de dominancia libres de sesgo mediático o humano.

Caso de estudio

El negocio de los deportes de combate, específicamente la Ultimate Fighting Championship (UFC), se basa en la competitividad y el espectáculo de los enfrentamientos. Para este proyecto, elegimos este tema debido a que el conjunto de datos disponible presentaba una calidad y nivel de detalle superiores en comparación con otras opciones analizadas. Este dataset está dividido en dos partes principales: datos a nivel de peleador y datos a nivel de combate. En los deportes de combate intervienen múltiples variables que pueden influir en el resultado de una pelea, generando un comportamiento complejo en los participantes. Tradicionalmente, el análisis deportivo se ha limitado a estadísticas descriptivas simples, por lo que el presente caso de estudio pretende ir más allá, definiendo cuatro problemas analíticos fundamentales para extraer patrones subyacentes.

Problema 1: Segmentación Automática de Perfiles Tácticos

El primer problema plantea la segmentación de perfiles tácticos (por ejemplo, Strikers contra Grapplers) para agrupar a los peleadores basándose estrictamente en su comportamiento numérico en la jaula. Esto se llevará a cabo a través de un análisis de agrupamiento (Clustering), utilizando el modelo K-Means. Las variables a procesar por el algoritmo incluyen el promedio de derribos, el promedio de golpes aterrizados y los intentos de sumisión. Este enfoque permitirá descubrir agrupaciones naturales de estilos de pelea sin depender de etiquetas asignadas manualmente.

Este enfoque se llevará a cabo mediante un análisis de **agrupamiento no supervisado**, utilizando específicamente el algoritmo **K-Means Clustering**, el cual agrupa a los peleadores minimizando la distancia euclidiana de cada uno al centroide de su grupo. Se identificarán las variables **avg_TD_landed**, **avg_SIG_STR_landed** y **avg_SUB_ATT** normalizadas por cada 15 minutos peleados como atributos del modelo, permitiendo descubrir agrupaciones naturales de estilos de pelea sin depender de etiquetas asignadas manualmente.

Problema 2: Determinantes de la Finalización

El segundo problema se centra en identificar qué combinación de variables físicas y técnicas disparan la probabilidad de un Knockout o una Sumisión. Para resolver esto, se realizará un análisis de clasificación utilizando algoritmos de Árboles de Decisión. Las variables independientes que se analizarán incluyen el alcance del peleador, la precisión de golpeo, la defensa de derribos y la categoría de peso. El objetivo del algoritmo será predecir nuestra variable objetivo, que es el método de finalización del combate (KO, Sumisión o Decisión).

Problema 3: El Impacto del Tiempo en el Atleta El tercer problema aborda el efecto de la edad en el declive del rendimiento físico de los peleadores, con el objetivo de modelar la pérdida de velocidad de golpeo y la capacidad de absorción de daño conforme avanza el tiempo. Este enfoque se llevará a cabo mediante un análisis predictivo de tendencia, utilizando específicamente una Regresión Polinomial de grado 2, la cual ajusta una curva matemática a los datos minimizando el Error Cuadrático Medio (MSE). Se utilizará la variable calculada **Age** como atributo independiente, mientras que las métricas continuas **SLpM** (golpes conectados por minuto) y **SApM** (golpes absorbidos por minuto) fungirán como variables dependientes, permitiendo determinar el vértice de máximo rendimiento atlético y su posterior declive.

Problema 4: Índice de Dominancia Multicriterio El último problema busca resolver la subjetividad en los rankings deportivos mediante la creación de un score numérico propio que clasifique a los peleadores más dominantes. Este enfoque se llevará a cabo mediante un análisis descriptivo y de optimización multicriterio, utilizando el algoritmo *Min-Max Scaling* para transformar métricas de diferentes dimensiones a una escala uniforme del 0 al 100. Se procesarán las variables calculadas **Net_Damage** (daño neto), **TD_Avg** (control de derribos) y **Finish_Rate** (tasa de finalización), aplicándoles pesos ponderados y una función matemática de decaimiento exponencial (*Time Decay*) basada en los años de inactividad del atleta, lo que permitirá generar un ranking algorítmico, imparcial y actualizado.

Descripción de técnicas

Problema 1 (Segmentación Automática de Perfiles Tácticos) : Agrupamiento (K-Means Clustering)

El agrupamiento o clustering es una técnica de aprendizaje no supervisado cuyo objetivo es descubrir estructura natural en los datos sin necesidad de etiquetas previas. El algoritmo K-Means divide el conjunto de observaciones en K grupos minimizando la distancia euclidiana de cada punto al centroide de su clúster, iterando hasta que las asignaciones se estabilizan. Para este problema se aplicó K-Means sobre tres variables tácticas normalizadas por cada 15 minutos peleados: golpes significativos conectados, derribos exitosos e intentos de sumisión. Previo al entrenamiento se aplicó estandarización Z-score mediante StandardScaler para evitar que la variable de mayor magnitud (golpes) dominará el cálculo de distancias. El número óptimo de clústeres se determinó combinando el Método del Codo, que evalúa la inercia para distintos valores de K , y el Silhouette Score, que mide la cohesión interna y separación entre grupos. Finalmente, los resultados se validaron visualmente mediante proyección PCA a dos dimensiones y cualitativamente comparando las asignaciones con el estilo conocido de peleadores reconocidos.

Problema 2 (Determinantes de la Finalización): Clasificación (Árboles de Decisión)

Cuando hablamos de clasificación en aprendizaje automático, nos referimos a una técnica que ayuda a las máquinas a reconocer patrones y a predecir a qué categoría específica pertenece algo, basándose en la información que ya tiene. Aquí, usamos un algoritmo en particular: los Árboles de Decisión. Nos pareció el mejor porque puede entender muy bien cómo se relacionan entre sí muchas cosas, como las habilidades físicas y las técnicas que usan los peleadores en un combate de artes marciales mixtas. La ventaja es que, a diferencia de otros modelos que son más complicados de descifrar, los árboles de decisión nos dan reglas que se entienden muy fácil. Así, podemos ver de forma clara y lógica qué cosas hacen que sea más probable que una pelea termine de cierta manera: por nocaut, por sumisión o por decisión de los jueces.

Problema 3 (El Impacto del Tiempo en el Atleta): Regresión No Lineal (Polinomial)

La regresión es un método estadístico y de aprendizaje automático predictivo que se utiliza para modelar la relación entre una variable dependiente (objetivo) y una o más variables independientes (predictoras), con el fin de pronosticar valores continuos. Para este problema en particular, se decidió utilizar una Regresión Polinomial en lugar de una regresión lineal simple. Esta decisión se tomó porque el rendimiento atlético humano no se deteriora en una línea recta perfecta; por el

contrario, presenta una curva de desarrollo que incluye una fase de ascenso en la juventud, un vértice de máximo rendimiento (peak físico) y un declive gradual. La regresión polinomial permite ajustar el algoritmo a esta forma curva (parábola), logrando predecir con mayor exactitud el volumen de golpes que un peleador conectará o absorberá en una edad específica.

Problema 4 (Índice de Dominancia Multicriterio): Análisis Descriptivo y Optimización Multicriterio

El análisis descriptivo multicriterio permite transformar, escalar e integrar múltiples variables heterogéneas en una sola métrica estandarizada. Se aplicará específicamente el algoritmo de normalización matemática *Min-Max Scaling*. Esta técnica es fundamental porque soluciona el problema de procesar métricas en diferentes unidades y dimensiones (como promedios de golpes por minuto frente a porcentajes de finalización), llevándolas todas a una escala uniforme del 0 al 100. Posteriormente, la técnica de optimización aplica pesos ponderados a estas variables normalizadas e integra una función matemática de penalización por inactividad (*Time Decay*). Este decaimiento exponencial reduce el puntaje de un atleta basándose en los años transcurridos desde su último combate, permitiendo rankear a los peleadores de forma objetiva, actual y sin sesgos mediáticos.

Tipo de análisis

El proyecto inicia con un **análisis de descubrimiento o no supervisado**, aplicado en la segmentación de perfiles, el cual explora la estructura interna de los datos para encontrar patrones ocultos sin una respuesta correcta previa. Posteriormente, se integran **análisis de tipo predictivo**, abarcando tanto la clasificación como la regresión; estos buscan proyectar comportamientos futuros, modelando matemáticamente el desgaste físico en el tiempo y anticipando las formas de finalización de un combate basándose en datos históricos. Por último, se complementa con un **análisis de tipo descriptivo y de optimización**, cuyo propósito es resumir, estandarizar y clasificar el rendimiento acumulado para crear un índice de dominancia actual y libre de sesgos. La combinación estratégica de estos enfoques garantiza una extracción de valor profunda y exhaustiva del caso de estudio.

Proceso de minería de datos

Problema 1: Segmentación Automática de Perfiles Tácticos (K-Means Clustering)

A. Algoritmos y herramientas

Para este análisis se utilizó Python como lenguaje principal, apoyado en las siguientes librerías: pandas y numpy para la manipulación y transformación de los datos, scikit-learn para el escalamiento (StandardScaler), el entrenamiento del modelo (KMeans), la reducción de dimensionalidad (PCA) y la validación (silhouette_score), y matplotlib junto con seaborn para la generación de todas las visualizaciones. La base de datos consultada fue el archivo **ufc_gold_dataset_final.csv**, obtenido de Kaggle, que contiene 8,551 combates registrados entre 1994 y 2026 con 37 columnas de estadísticas por pelea.

B. Recolección de datos

Los datos fueron extraídos del dataset público de UFC disponible en Kaggle. Para este análisis se utilizó específicamente el archivo de combates debido a que contiene las estadísticas reales de cada peleador dentro del octágono. Dado que el dataset registra una fila por combate con prefijos **F1** y **F2** para cada participante, se realizó un proceso de apilado para convertir cada combate en dos filas, obteniendo así una vista a nivel de peleador. Posteriormente se agregaron los totales acumulados de carrera y se normalizaron las estadísticas por cada 15 minutos peleados, que es el estándar oficial de UFC Stats, aplicando la fórmula:

$$feature_per15 = (total_feature / total_time_sec) \times 900$$

C. Imagen del dataset y atributos

```
In [3]: # Vista previa: primeras filas con las columnas mas relevantes para el analisis
columns_preview = [
    "Fighter_1", "Fighter_2", "Winner", "Weight_Class", "Method",
    "Total_Fight_Time_Sec",
    "F1_Sig_Landed", "F1_TD_Landed", "F1_Sub_Att",
    "F2_Sig_Landed", "F2_TD_Landed", "F2_Sub_Att",
    "Event_Date",
]
df_fights[columns_preview].head(10)
```

	Fighter_1	Fighter_2	Winner	Weight_Class	Method	Total_Fight_Time_Sec	F1_Sig_Landed	F1_TD_Landed	F1_Sub_Att	F2_Sig_Landed	F2_TD_Landed	F2_Sub_Att	Event_Date
0	Scott Morris	Sean Daugherty	Scott Morris	Open Weight Bout	Submission	20	1	1	1	0	0	0	1994-03-11
1	Patrick Smith	Ray Wizard	Patrick Smith	Open Weight Bout	Submission	58	1	0	1	1	0	0	1994-03-11
2	Royce Gracie	Patrick Smith	Royce Gracie	UFC 2 Tournament Title Bout	KO/TKO	77	4	1	0	1	0	0	1994-03-11
3	Royce Gracie	Remco Pardoel	Royce Gracie	Open Weight Bout	Submission	91	0	1	1	0	0	0	1994-03-11
4	Patrick Smith	Johnny Rhodes	Patrick Smith	Open Weight Bout	Submission	67	5	0	1	4	0	0	1994-03-11
5	Royce Gracie	Jason DeLucia	Royce Gracie	Open Weight Bout	Submission	67	0	0	1	0	1	0	1994-03-11
6	Remco Pardoel	Orlando Wiet	Remco Pardoel	Open Weight Bout	KO/TKO	89	7	1	0	1	0	0	1994-03-11
7	Johnny Rhodes	Fred Ettish	Johnny Rhodes	Open Weight Bout	Submission	187	13	0	1	4	0	0	1994-03-11
8	Royce Gracie	Minoki Ichihara	Royce Gracie	Open Weight Bout	Submission	308	2	1	2	3	0	0	1994-03-11
9	Jason DeLucia	Scott Baker	Jason DeLucia	Open Weight Bout	Submission	401	3	0	5	0	1	0	1994-03-11

La imagen anterior muestra una vista previa del dataset de combates utilizado como fuente principal del análisis. Cada fila representa un combate registrado e incluye los nombres de ambos participantes, el ganador, la categoría de peso, el método de finalización, el tiempo total de la pelea en segundos y las estadísticas individuales de cada peleador como golpes significativos conectados, derribos exitosos e intentos de sumisión. Estas últimas tres columnas, junto con el tiempo total peleado, son la base para construir las variables normalizadas que alimentan el modelo de clustering.

D. Pasos del análisis

Paso 1: Apilado y agregación de datos por peleador

```
In [10]: # PASO 1: Apilar F1 y F2 en formato largo
# Cada combate genera dos filas: una para cada peleador participante

# Vista del peleador 1
df_f1 = df_fights[[
    "Fighter_1", "Total_Fight_Time_Sec",
    "F1_Sig_Landed", "F1_TD_Landed", "F1_Sub_Att", "F1_Ctrl_Sec", "F1_KD",
]].rename(columns={
    "Fighter_1": "Fighter",
    "Total_Fight_Time_Sec": "Fight_Time_Sec",
    "F1_Sig_Landed": "Sig_Landed",
    "F1_TD_Landed": "TD_Landed",
    "F1_Sub_Att": "Sub_Att",
    "F1_Ctrl_Sec": "Ctrl_Sec",
    "F1_KD": "KD",
})

# Vista del peleador 2
df_f2 = df_fights[[
    "Fighter_2", "Total_Fight_Time_Sec",
    "F2_Sig_Landed", "F2_TD_Landed", "F2_Sub_Att", "F2_Ctrl_Sec", "F2_KD",
]].rename(columns={
    "Fighter_2": "Fighter",
    "Total_Fight_Time_Sec": "Fight_Time_Sec",
    "F2_Sig_Landed": "Sig_Landed",
    "F2_TD_Landed": "TD_Landed",
    "F2_Sub_Att": "Sub_Att",
    "F2_Ctrl_Sec": "Ctrl_Sec",
    "F2_KD": "KD",
})

# Concatenamos verticalmente: el resultado tiene 2x el numero de combates
df_stacked = pd.concat([df_f1, df_f2], ignore_index=True)

print(f"Combates originales: {len(df_fights):,}")
print(f"Filas tras apilar: {len(df_stacked):,} (= 2 x combates)")
print(f"\nPrimeras filas del DataFrame apilado:")
df_stacked.head()
```

Combates originales: 8,551
Filas tras apilar: 17,102 (= 2 x combates)

Primeras filas del DataFrame apilado:

```
Out[10]:
```

	Fighter	Fight_Time_Sec	Sig_Landed	TD_Landed	Sub_Att	Ctrl_Sec	KD
0	Scott Morris	20	1	1	1	0	0
1	Patrick Smith	58	1	0	1	0	0
2	Royce Gracie	77	4	1	0	0	0
3	Royce Gracie	91	0	1	1	0	0
4	Patrick Smith	67	5	0	1	0	0

Dado que el dataset original registra una fila por combate con estadísticas separadas para cada participante, fue necesario transformarlo a un formato donde cada fila representa a un peleador. Para esto se apilaron las vistas de F1 y F2 verticalmente, generando el doble de filas, y posteriormente se agruparon por nombre de peleador sumando los totales acumulados de su carrera. Se filtraron únicamente los peleadores con al menos 6 combates UFC para garantizar que las estadísticas fueran representativas, resultando en una muestra final de 1,093 peleadores.

Paso 2: Normalización por 15 minutos y escalamiento

```
In [12]: # PASO 3: Normalizar por 15 minutos peleados (estandar UFC Stats)
# Formula: feature_per15 = total_feature / total_time_sec * 900 (900 seg = 15 min)
SEGUNDOS_15_MIN = 900

df_fighters_agg["avg_SIG_landed_per15"] = (
    df_fighters_agg["total_sig"] / df_fighters_agg["total_time_sec"] * SEGUNDOS_15_MIN
)
df_fighters_agg["avg_TD_landed_per15"] = (
    df_fighters_agg["total_td"] / df_fighters_agg["total_time_sec"] * SEGUNDOS_15_MIN
)
df_fighters_agg["avg_SUB_att_per15"] = (
    df_fighters_agg["total_sub"] / df_fighters_agg["total_time_sec"] * SEGUNDOS_15_MIN
)
# Features adicionales para el analisis bonus
df_fighters_agg["avg_CTRL_sec_per15"] = (
    df_fighters_agg["total_ctrl"] / df_fighters_agg["total_time_sec"] * SEGUNDOS_15_MIN
)
df_fighters_agg["avg_KD_per15"] = (
    df_fighters_agg["total_kd"] / df_fighters_agg["total_time_sec"] * SEGUNDOS_15_MIN
)

# Vista previa con las columnas relevantes
columnas_vista = [
    "Fighter", "num_fights",
    "avg_SIG_landed_per15", "avg_TD_landed_per15", "avg_SUB_att_per15",
    "avg_CTRL_sec_per15", "avg_KD_per15",
]
print("DataFrame a nivel peleador (primeros 10 registros, ordenados por numero de peleas):\n")
df_fighters_agg[columnas_vista].sort_values("num_fights", ascending=False).head(10).round(2)
```

DataFrame a nivel peleador (primeros 10 registros, ordenados por numero de peleas):

Out[12]:

	Fighter	num_fights	avg_SIG_landed_per15	avg_TD_landed_per15	avg_SUB_att_per15	avg_CTRL_sec_per15	avg_KD_per15
1181	Jim Miller	46	43.82	1.50	1.70	188.62	0.20
145	Andrei Arlovski	42	57.99	0.33	0.18	70.47	0.37
711	Donald Cerrone	38	73.49	1.01	0.46	97.37	0.84
493	Clay Guida	37	40.97	2.76	0.57	332.65	0.21
1880	Neil Magny	37	51.01	2.17	0.27	192.34	0.07
420	Charles Oliveira	37	48.50	2.29	2.59	311.30	0.41
2025	Rafael Dos Anjos	36	52.22	1.95	0.57	235.18	0.20
1153	Jeremy Stephens	35	47.31	1.11	0.37	158.85	0.67
660	Demian Maia	33	27.06	2.49	0.99	341.22	0.04
689	Diego Sanchez	32	39.96	1.36	0.68	225.20	0.23

```
In [18]: # Aplicar StandardScaler a las 3 features principales
X = df_fighters[FEATURES_PRINCIPALES].values

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Convertir a DataFrame para facilitar la inspeccion
df_scaled = pd.DataFrame(X_scaled, columns=FEATURES_PRINCIPALES)

print("Estadísticas DESPUES del escalamiento (deben tener media ~0 y std ~1):\n")
df_scaled.describe().round(3)
```

Estadísticas DESPUES del escalamiento (deben tener media ~0 y std ~1):

Out[18]:

	avg_SIG_landed_per15	avg_TD_landed_per15	avg_SUB_att_per15
count	1093.000	1093.000	1093.000
mean	-0.000	0.000	0.000
std	1.000	1.000	1.000
min	-2.576	-1.249	-0.942
25%	-0.719	-0.753	-0.733
50%	-0.067	-0.242	-0.270
75%	0.640	0.496	0.431
max	4.314	4.460	6.777

Una vez agregados los totales de carrera por peleador, se normalizaron las estadísticas dividiéndolas entre el tiempo total peleado en segundos y multiplicándose por 900, equivalente a 15 minutos, que es el estándar oficial de UFC Stats. Esto permite comparar de forma justa a peleadores con carreras de distinta duración. Posteriormente se aplicó un escalamiento Z-score mediante StandardScaler para que las tres variables quedarán en la misma escala y el algoritmo K-Means no se sesgara hacia los golpes significativos, que tienen una magnitud mucho mayor que los derribos y los intentos de sumisión.

Paso 3: Selección del número óptimo de clústeres

```
In [20]: # Calcular inercia y silhouette score para K = 2 hasta 10
K_range = range(2, 11)
inercias = []
silhouettes = []

for k in K_range:
    km = KMeans(n_clusters=k, random_state=RANDOM_STATE, n_init=10)
    labels = km.fit_predict(X_scaled)
    inercias.append(km.inertia_)
    silhouettes.append(silhouette_score(X_scaled, labels))
    print(f"K={k:2d} | Inercia = {km.inertia_:8.2f} | Silhouette = {silhouettes[-1]:.4f}")

K= 2 | Inercia = 2140.69 | Silhouette = 0.3470
K= 3 | Inercia = 1693.59 | Silhouette = 0.3486
K= 4 | Inercia = 1303.40 | Silhouette = 0.2991
K= 5 | Inercia = 1164.95 | Silhouette = 0.2738
K= 6 | Inercia = 1050.17 | Silhouette = 0.2610
K= 7 | Inercia = 965.51 | Silhouette = 0.2650
K= 8 | Inercia = 867.10 | Silhouette = 0.2570
K= 9 | Inercia = 806.39 | Silhouette = 0.2536
K=10 | Inercia = 756.25 | Silhouette = 0.2613
```

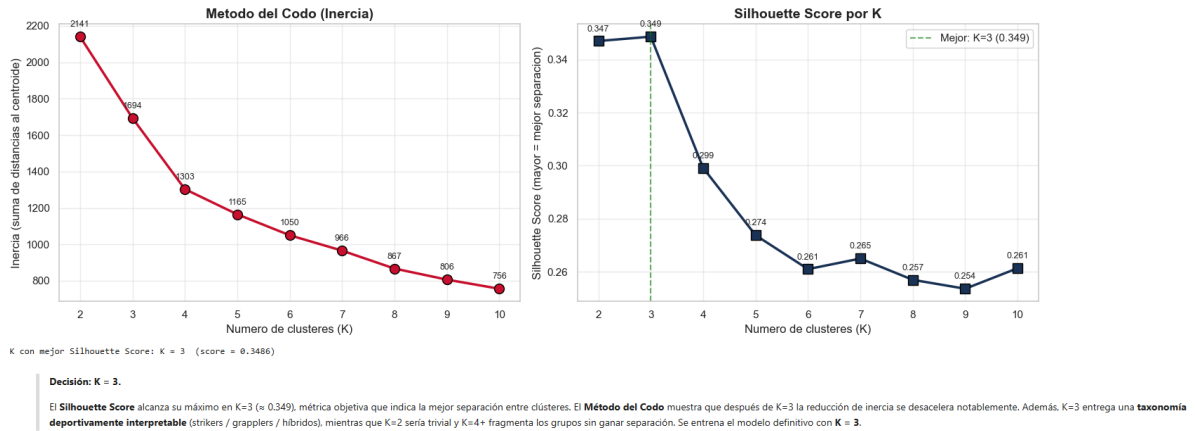
```
In [21]: # Visualizacion del Metodo del Codo y Silhouette Score lado a lado
fig, axes = plt.subplots(1, 2, figsize=(15, 5))

# --- Metodo del Codo ---
axes[0].plot(list(K_range), inercias, marker="o", linewidth=2.5,
             markersize=10, color="#C8102E", markeredgcolor="black")
axes[0].set_title("Metodo del Codo (Inercia)", fontsize=14, fontweight="bold")
axes[0].set_xlabel("Numero de clusteres (K)")
axes[0].set_ylabel("Inercia (suma de distancias al centroide)")
axes[0].set_xticks(list(K_range))
axes[0].grid(alpha=0.3)
for k, val in zip(K_range, inercias):
    axes[0].annotate(f"{val:.0f}", (k, val), textcoords="offset points",
                    xytext=(0, 10), ha="center", fontsize=9)

# --- Silhouette Score ---
axes[1].plot(list(K_range), silhouettes, marker="s", linewidth=2.5,
             markersize=10, color="#1D3557", markeredgcolor="black")
axes[1].set_title("Silhouette Score por K", fontsize=14, fontweight="bold")
axes[1].set_xlabel("Numero de clusteres (K)")
axes[1].set_ylabel("Silhouette Score (mayor = mejor separacion)")
axes[1].set_xticks(list(K_range))
axes[1].grid(alpha=0.3)
# Resaltar el K con mejor silhouette
k_mejor_sil = list(K_range)[np.argmax(silhouettes)]
sil_max = max(silhouettes)
axes[1].axvline(k_mejor_sil, color="green", linestyle="--", alpha=0.6,
                label=f"Mejor: K={k_mejor_sil} ({sil_max:.3f})")
axes[1].legend()
for k, val in zip(K_range, silhouettes):
    axes[1].annotate(f"{val:.3f}", (k, val), textcoords="offset points",
                    xytext=(0, 10), ha="center", fontsize=9)

plt.tight_layout()
plt.show()

print(f"\nK con mejor Silhouette Score: K = {k_mejor_sil} (score = {sil_max:.4f})")
```



Para determinar el valor óptimo de K se evaluaron dos métricas complementarias para valores de K del 2 al 10. La primera es la Inercia, medida mediante el Método del Codo, que calcula la suma de distancias cuadradas de cada punto al centroide de su clúster; se busca el punto donde la reducción de inercia comienza a aplanarse. La segunda es el Silhouette Score, que mide qué tan bien separado está cada punto de su propio clúster respecto al clúster vecino más cercano, con valores entre -1 y 1 donde valores más altos indican mejor separación.

Como resultado, se seleccionó **K = 3** debido a que el Silhouette Score alcanzó su valor máximo en ese punto y el Método del Codo mostró una desaceleración notable en la reducción de inercia a partir de K = 3, además de que tres clústeres entrega una taxonomía deportivamente interpretable como strikers, wrestlers y grapplers.

Paso 4: Entrenamiento del modelo K-Means

```
In [22]: # Entrenamiento del K-Means definitivo
K_FINAL = 3
kmeans = KMeans(n_clusters=K_FINAL, random_state=RANDOM_STATE, n_init=10)
kmeans.fit(X_scaled)

# Asignar la etiqueta de cluster a cada peleador
df_fighters["Cluster"] = kmeans.labels_

# Recuperar los centroides en la ESCALA ORIGINAL (mas interpretables que en Z-score)
centroides_originales = scaler.inverse_transform(kmeans.cluster_centers_)
df_centroides = pd.DataFrame(centroides_originales, columns=FEATURES_PRINCIPALES)
df_centroides.index.name = "Cluster"

print("Centroides (en escala original, valores por 15 min de pelea):\n")
df_centroides.round(2)
```

Centroides (en escala original, valores por 15 min de pelea):

```
Out[22]:
```

Cluster	avg_SIG_landed_per15	avg_TD_landed_per15	avg_SUB_att_per15
0	60.84	0.87	0.31
1	37.04	1.61	1.66
2	43.89	3.14	0.59

```
In [23]: # Distribucion de peleadores por cluster
conteo = df_fighters["Cluster"].value_counts().sort_index()
porcentaje = (conteo / len(df_fighters) * 100).round(1)

resumen = pd.DataFrame({
    "Cantidad de peleadores": conteo,
    "Porcentaje (%)": porcentaje,
})
print("Distribucion de peleadores por cluster:\n")
print(resumen)
print(f"\nInercia final del modelo: {kmeans.inertia_.2f}")
print(f"Silhouette Score: {silhouette_score(X_scaled, kmeans.labels_):.4f}")
```

Distribucion de peleadores por cluster:

Cluster	Cantidad de peleadores	Porcentaje (%)
0	653	59.7
1	166	15.2
2	274	25.1

Inercia final del modelo: 1693.59

Silhouette Score: 0.3486

Se entrenó el modelo K-Means definitivo con $K = 3$, semilla aleatoria fija de 42 para garantizar reproducibilidad y 10 inicializaciones diferentes para quedarse con la solución de menor inercia. Los centroides obtenidos se convirtieron de vuelta a la escala original para facilitar su interpretación deportiva, confirmando los tres perfiles tácticos: Strikers puros con aproximadamente 61 golpes por 15 minutos, Wrestlers con 3.14 derribos por 15 minutos y Grapplers ofensivos con 1.66 intentos de sumisión por 15 minutos.

Paso 5: Visualizaciones de los clústeres

```
In [25]: # Visualizacion 1: Scatter 3D con Los 3 features y centroides marcados
fig = plt.figure(figsize=(14, 15))
ax = fig.add_subplot(111, projection="3d")

for c in sorted(df_fighters["Cluster"].unique()):
    sub = df_fighters[df_fighters["Cluster"] == c]
    ax.scatter(
        sub["avg_SIG_landed_per15"],
        sub["avg_TD_landed_per15"],
        sub["avg_SUB_att_per15"],
        c=COLORES_CLUSTER[c],
        label=f"{NOMBRES_CLUSTER[c]} (n={len(sub)}",
        alpha=0.55,
        s=35,
        edgecolors="black",
        linewidths=0.3,
    )

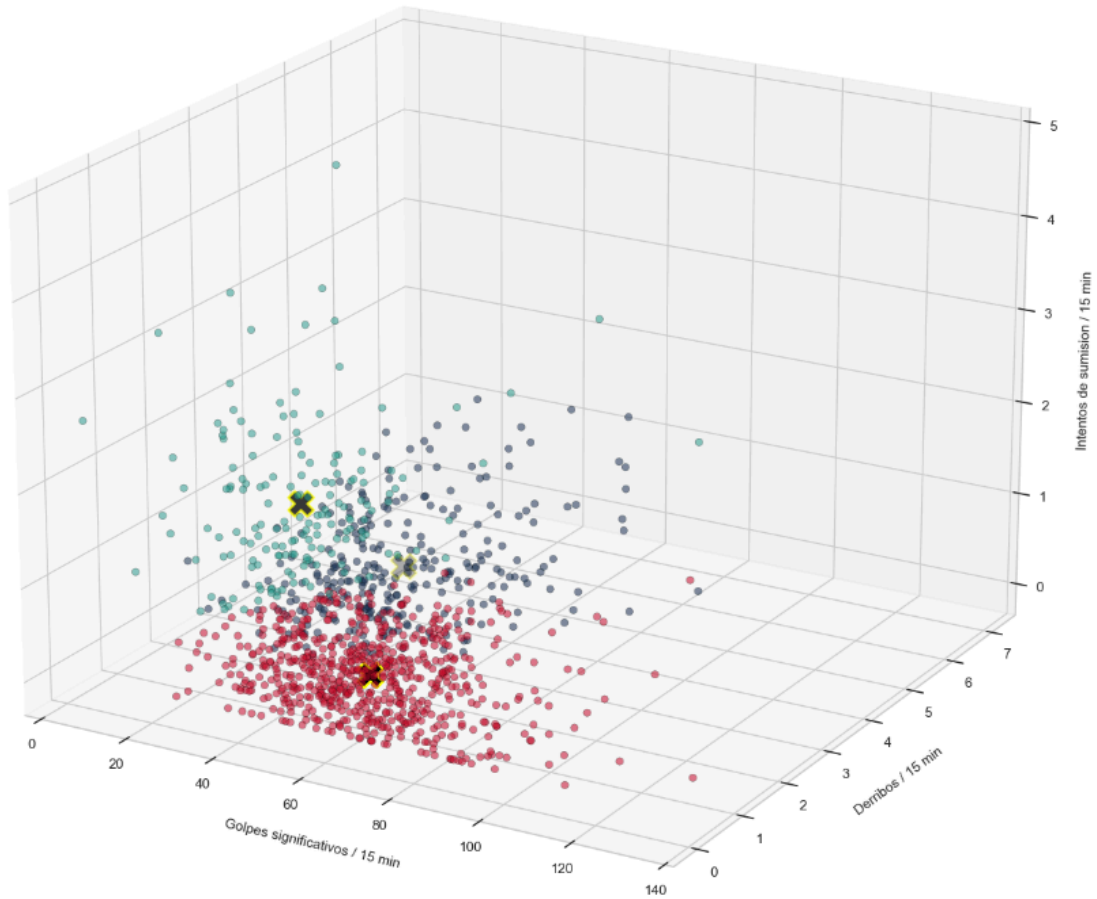
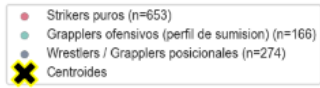
# Marcar Los centroides como X grandes con borde amarillo
ax.scatter(
    centroides_originales[:, 0],
    centroides_originales[:, 1],
    centroides_originales[:, 2],
    c="black",
    marker="X",
    s=400,
    edgecolors="yellow",
    linewidths=2,
    label="Centroides",
)

ax.set_xlabel("Golpes significativos / 15 min", fontsize=11, labelpad=12)
ax.set_ylabel("Derribos / 15 min", fontsize=11, labelpad=12)
ax.set_zlabel("Intentos de sumision / 15 min", fontsize=11, labelpad=10)
ax.set_title("Clusteres de estilos tacticos en el espacio 3D",
             fontsize=15, fontweight="bold", pad=30)
ax.legend(loc="upper left", fontsize=10, framealpha=0.9, bbox_to_anchor=(0.02, 0.98))

# Ajustar el angulo de la vista para que se vean los tres ejes claramente
ax.view_init(elev=20, azim=-60)

# Margenes manuales para evitar que el titulo o ejes se corten
plt.subplots_adjust(left=0.05, right=0.95, top=0.92, bottom=0.05)
plt.show()
```

Clústeres de estilos tácticos en el espacio 3D



```
In [26]: # Visualización 2: Proyección PCA 2D
# PCA reduce las 3 features a 2 componentes principales que conservan la mayor varianza
pca = PCA(n_components=2, random_state=RANDOM_STATE)
X_pca = pca.fit_transform(X_scaled)

# También proyectamos los centroides al espacio PCA
centroides_pca = pca.transform(kmeans.cluster_centers_)

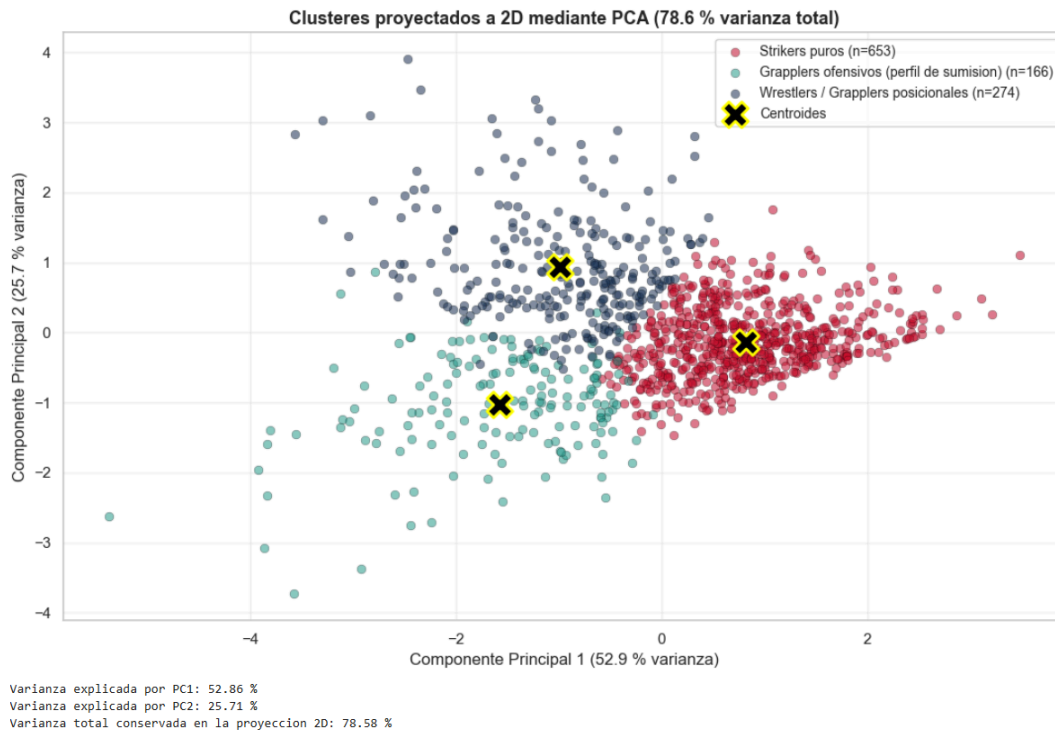
# Porcentaje de varianza explicada por cada componente
varianza = pca.explained_variance_ratio_ * 100

fig, ax = plt.subplots(figsize=(11, 7))
for c in sorted(df_fighters["Cluster"].unique()):
    mask = df_fighters["Cluster"] == c
    ax.scatter(
        X_pca[mask, 0], X_pca[mask, 1],
        c=COLORES_CLUSTER[c],
        label=f"NOMBRES_CLUSTER[c] (n={mask.sum()})",
        alpha=0.55, s=40, edgecolors="black", linewidths=0.3,
    )

# Centroides en el espacio PCA
ax.scatter(
    centroides_pca[:, 0], centroides_pca[:, 1],
    c="black", marker="X", s=350,
    edgecolors="yellow", linewidths=2, label="Centroides",
)

ax.set_xlabel(f"Componente Principal 1 ({varianza[0]:.1f} % varianza)", fontsize=12)
ax.set_ylabel(f"Componente Principal 2 ({varianza[1]:.1f} % varianza)", fontsize=12)
ax.set_title(f"Clústeres proyectados a 2D mediante PCA "
             f"({varianza.sum():.1f} % varianza total)",
             fontsize=13, fontweight="bold")
ax.legend(loc="best", fontsize=10, framealpha=0.9)
ax.grid(alpha=0.3)
plt.tight_layout()
plt.show()

print(f"Varianza explicada por PC1: {varianza[0]:.2f} %")
print(f"Varianza explicada por PC2: {varianza[1]:.2f} %")
print(f"Varianza total conservada en la proyección 2D: {varianza.sum():.2f} %")
```



```
In [27]: # Visualizacion 3: Radar chart de Los centroides
# Compara el "perfil" de los 3 clusteres en una sola figura
from math import pi

# Normalizar los centroides al rango 0-1 para que el radar sea legible
cent_min = centroides_originales.min(axis=0)
cent_max = centroides_originales.max(axis=0)
centroides_norm = (centroides_originales - cent_min) / (cent_max - cent_min)

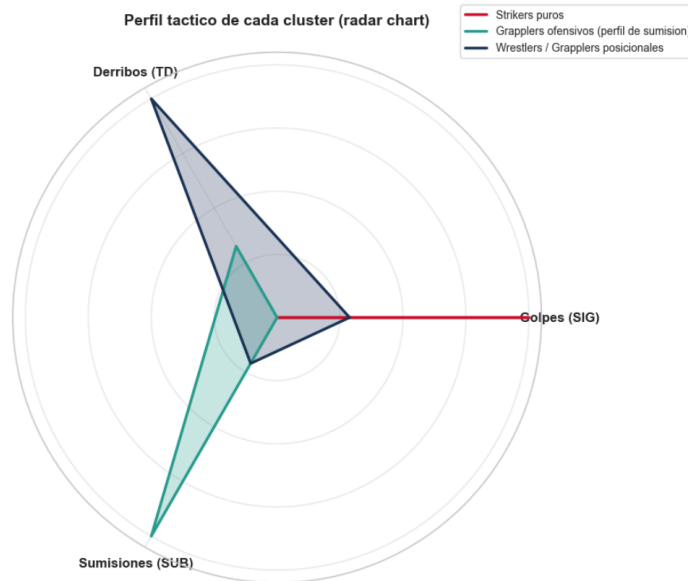
# Etiquetas en español para los ejes del radar
etiquetas = ["Golpes (SIG)", "Derribos (TD)", "Sumisiones (SUB)"]
N = len(etiquetas)

# Calcular angulos del poligono
angulos = [n / float(N) * 2 * pi for n in range(N)]
angulos += angulos[:1] # cerrar el poligono

fig, ax = plt.subplots(figsize=(9, 9), subplot_kw=dict(polar=True))

for c in sorted(df_fighters["Cluster"].unique()):
    valores = centroides_norm[c].tolist()
    valores += valores[:1] # cerrar el poligono
    ax.plot(angulos, valores, color=COLORES_CLUSTER[c], linewidth=2.5,
            label=NOMBRES_CLUSTER[c])
    ax.fill(angulos, valores, color=COLORES_CLUSTER[c], alpha=0.25)

ax.set_xticks(angulos[:-1])
ax.set_xticklabels(etiquetas, fontsize=12, fontweight="bold")
ax.set_yticks([0.25, 0.5, 0.75, 1.0])
ax.set_yticklabels(["", "", "", ""]) # ocultamos los valores radiales (la forma es lo que importa)
ax.set_ylim(0, 1.05)
ax.set_title("Perfil tactico de cada cluster (radar chart)",
             fontsize=14, fontweight="bold", pad=25)
ax.legend(loc="upper right", bbox_to_anchor=(1.3, 1.1), fontsize=10)
ax.grid(alpha=0.4)
plt.tight_layout()
plt.show()
```



```
In [28]: # Visualizacion 4: Boxplots de cada feature segmentados por cluster
# Validacion visual de que la separacion entre grupos es real para CADA variable
fig, axes = plt.subplots(1, 3, figsize=(16, 5))
```

```
nombres_legibles = {
    "avg_SIG_landed_per15": "Golpes significativos / 15 min",
    "avg_TD_landed_per15": "Derribos / 15 min",
    "avg_SUB_att_per15": "Intentos de sumision / 15 min",
}

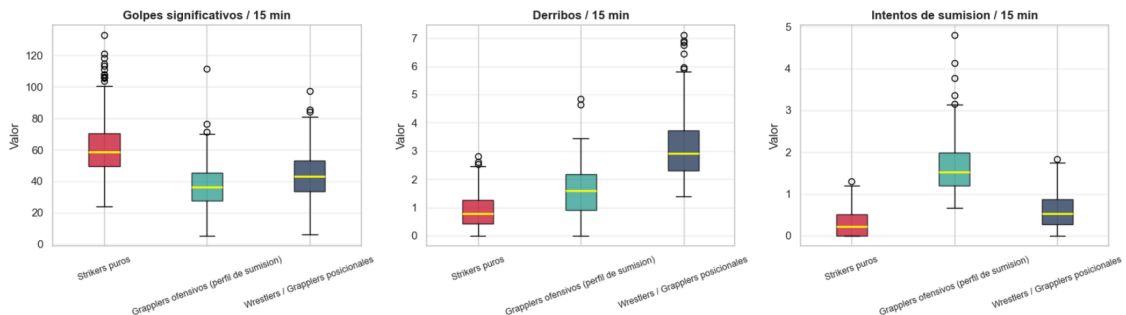
for i, feat in enumerate(FEATURES_PRINCIPALES):
    datos_por_cluster = [df_fighters[df_fighters["Cluster"] == c][feat].values
                        for c in sorted(df_fighters["Cluster"].unique())]
    nombres = [NOMBRES_CLUSTER[c] for c in sorted(df_fighters["Cluster"].unique())]
    colores = [COLORES_CLUSTER[c] for c in sorted(df_fighters["Cluster"].unique())]

    bp = axes[i].boxplot(datos_por_cluster, labels=nombres, patch_artist=True,
                        medianprops={"color": "yellow", "linewidth": 2})
    for patch, color in zip(bp["boxes"], colores):
        patch.set_facecolor(color)
        patch.set_alpha(0.75)

    axes[i].set_title(nombres_legibles[feat], fontsize=12, fontweight="bold")
    axes[i].set_ylabel("Valor")
    axes[i].tick_params(axis="x", rotation=20, labelsize=9)
    axes[i].grid(axis="y", alpha=0.3)

plt.suptitle("Distribucion de cada feature por cluster",
             fontsize=14, fontweight="bold", y=1.02)
plt.tight_layout()
plt.show()
```

Distribucion de cada feature por cluster



Cómo leer estas gráficas (mini-guía):

- Cada **caja** contiene el 50% central de los peleadores de ese cluster; la **línea amarilla** es la mediana (el peleador "típico" del grupo).
- Los **puntos sueltos** por encima son outliers: peleadores excepcionales que llevan al extremo el estilo de su cluster.
- En cada panel, busca **qué caja está más arriba**: esa identifica al cluster especialista en esa habilidad. Las cajas deben verse "escalonadas" (no traslapadas) para confirmar que la separación entre grupos es real.

Interpretación (Boxplots por cluster): confirman que la separación es **real para cada variable individualmente**, no solo en el espacio combinado. En cada gráfica, el cluster asociado a esa habilidad muestra una caja claramente elevada respecto a los otros dos: los strikers dominan en golpes, los sumisores en intentos de sumisión y los wrestlers en derribos. La presencia de outliers indica peleadores excepcionales que llevan al extremo el estilo de su cluster (potenciales casos icónicos como Khabib o Demian Maia).

Se generaron cuatro visualizaciones complementarias para validar y comunicar los resultados del clustering. El scatter 3D muestra las tres nubes de puntos separadas en el espacio tridimensional de las variables. La proyección PCA 2D reduce las dimensiones conservando la mayor varianza posible y confirma la separación entre grupos de forma más limpia. El radar chart permite comparar visualmente el perfil táctico de los tres clústeres en una sola figura, mostrando en qué variable destaca cada arquetipo. Finalmente los boxplots confirman que la separación entre grupos es real para cada variable individualmente.

Paso 6: Validación cualitativa con peleadores famosos

```
In [31]: # Lista curada de peleadores famosos con su estilo esperado
PELEADORES_FAMOSOS = {
    # Strikers
    "Conor McGregor": "Strikers puros",
    "Israel Adesanya": "Strikers puros",
    "Max Holloway": "Strikers puros",
    "Alex Pereira": "Strikers puros",
    "Stipe Miocic": "Strikers puros",
    "Robert Whittaker": "Strikers puros",
    # Wrestlers
    "Khabib Nurmagomedov": "Wrestlers / Grapplers posicionales",
    "Kamaru Usman": "Wrestlers / Grapplers posicionales",
    "Islam Makhachev": "Wrestlers / Grapplers posicionales",
    "Henry Cejudo": "Wrestlers / Grapplers posicionales",
    "Daniel Cormier": "Wrestlers / Grapplers posicionales",
    # Grapplers ofensivos
    "Charles Oliveira": "Grapplers ofensivos (perfil de sumision)",
    "Demian Maia": "Grapplers ofensivos (perfil de sumision)",
    "Brian Ortega": "Grapplers ofensivos (perfil de sumision)",
    "Tony Ferguson": "Grapplers ofensivos (perfil de sumision)",
}
```

```
# Buscar cada peleador en el DataFrame y comparar con su estilo esperado
filas_validacion = []
for nombre, estilo_esperado in PELEADORES_FAMOSOS.items():
    row = df_fighters[df_fighters["Fighter"] == nombre]
    if row.empty:
        filas_validacion.append({
            "Peleador": nombre,
            "Estilo esperado": estilo_esperado,
            "Cluster asignado": "(no encontrado)",
            "SIG/15": None, "TD/15": None, "SUB/15": None,
            "Coincide": "N/A",
        })
        continue
    r = row.iloc[0]
    asignado = r["Cluster_Nombre"]
    filas_validacion.append({
        "Peleador": nombre,
        "Estilo esperado": estilo_esperado,
        "Cluster asignado": asignado,
        "SIG/15": round(r["avg_SIG_landed_per15"], 1),
        "TD/15": round(r["avg_TD_landed_per15"], 2),
        "SUB/15": round(r["avg_SUB_att_per15"], 2),
        "Coincide": "Si" if asignado == estilo_esperado else "No",
    })

df_validacion = pd.DataFrame(filas_validacion)
```

```
# Estilo visual de la tabla: colorear filas segun coincidencia
def color_coincide(val):
    if val == "Si":
        return "background-color: #d4edda; color: #155724; font-weight: bold;"
    if val == "No":
        return "background-color: #f8d7da; color: #721c24; font-weight: bold;"
    return "background-color: #fff3cd; color: #856404;"

(df_validacion.style
 .map(color_coincide, subset=["Coincide"])
 .set_properties(**{"text-align": "left"})
 .set_table_styles([
     {"selector": "th", "props": [("background-color", "#1D3557"),
                                   ("color", "white"),
                                   ("font-weight", "bold"),
                                   ("text-align", "left")]},
 ])
))
```

Out[31]:

	Peleador	Estilo esperado	Cluster asignado	SIG/15	TD/15	SUB/15	Coincide
0	Conor McGregor	Strikers puros	Strikers puros	79.800000	0.670000	0.130000	Si
1	Israel Adesanya	Strikers puros	Strikers puros	60.200000	0.050000	0.140000	Si
2	Max Holloway	Strikers puros	Strikers puros	103.600000	0.230000	0.280000	Si
3	Alex Pereira	Strikers puros	Strikers puros	77.400000	0.110000	0.230000	Si
4	Stipe Miocic	Strikers puros	Strikers puros	70.000000	1.740000	0.000000	Si
5	Robert Whittaker	Strikers puros	Strikers puros	65.900000	0.730000	0.000000	Si
6	Khabib Nurmagomedov	Wrestlers / Grapplers posicionales	Wrestlers / Grapplers posicionales	61.500000	5.320000	0.790000	Si
7	Kamaru Usman	Wrestlers / Grapplers posicionales	Wrestlers / Grapplers posicionales	62.400000	2.790000	0.090000	Si
8	Islam Makhachev	Wrestlers / Grapplers posicionales	Wrestlers / Grapplers posicionales	36.700000	3.100000	0.980000	Si
9	Henry Cejudo	Wrestlers / Grapplers posicionales	Strikers puros	57.500000	1.780000	0.140000	No
10	Daniel Cormier	Wrestlers / Grapplers posicionales	Strikers puros	65.900000	1.660000	0.580000	No
11	Charles Oliveira	Grapplers ofensivos (perfil de sumision)	Grapplers ofensivos (perfil de sumision)	48.500000	2.290000	2.590000	Si
12	Demian Maia	Grapplers ofensivos (perfil de sumision)	Wrestlers / Grapplers posicionales	27.100000	2.490000	0.990000	No
13	Brian Ortega	Grapplers ofensivos (perfil de sumision)	Strikers puros	57.600000	0.930000	0.930000	No
14	Tony Ferguson	Grapplers ofensivos (perfil de sumision)	Strikers puros	73.600000	0.390000	0.880000	No

```

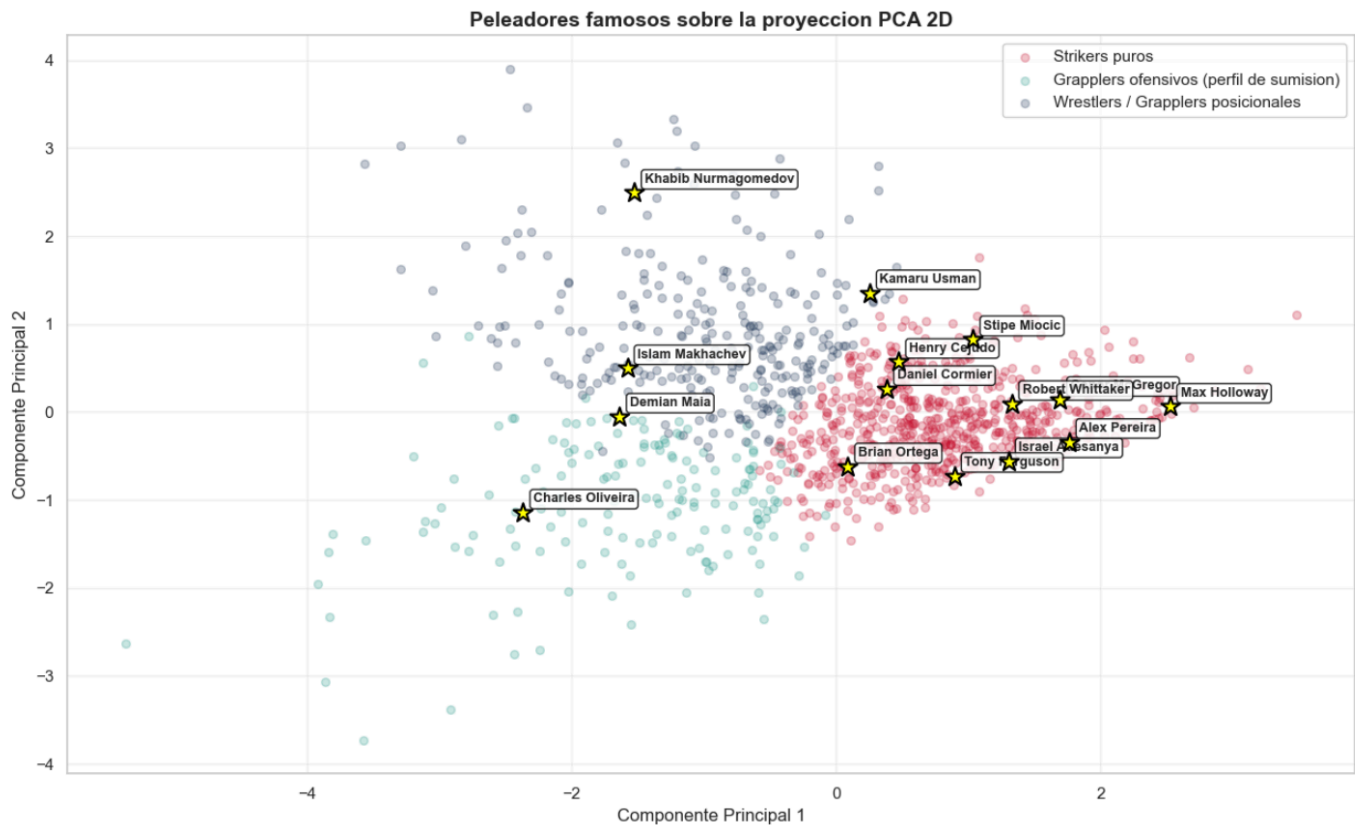
In [33]: # Visualizar a los peleadores famosos sobre la proyeccion PCA 2D
fig, ax = plt.subplots(figsize=(13, 8))

# Fondo: todos los peleadores coloreados por cluster (semi-transparentes)
for c in sorted(df_fighters["Cluster"].unique()):
    mask = df_fighters["Cluster"] == c
    ax.scatter(X_pca[mask, 0], X_pca[mask, 1],
               c=COLORES_CLUSTER[c], alpha=0.25, s=30,
               label=NOMBRES_CLUSTER[c])

# Foreground: peleadores famosos resaltados con etiqueta
for nombre in PELEADORES_FAMOSOS.keys():
    idx_arr = df_fighters.index[df_fighters["Fighter"] == nombre].tolist()
    if not idx_arr:
        continue
    idx = idx_arr[0]
    x, y = X_pca[idx, 0], X_pca[idx, 1]
    ax.scatter(x, y, c="yellow", s=180, marker="*",
               edgecolors="black", linewidths=1.5, zorder=5)
    ax.annotate(nombre, (x, y), xytext=(7, 7),
                textcoords="offset points", fontsize=9,
                fontweight="bold",
                bbox=dict(boxstyle="round,pad=0.25",
                        facecolor="white", edgecolor="black", alpha=0.85))

ax.set_xlabel("Componente Principal 1", fontsize=12)
ax.set_ylabel("Componente Principal 2", fontsize=12)
ax.set_title("Peleadores famosos sobre la proyeccion PCA 2D",
             fontsize=14, fontweight="bold")
ax.legend(loc="upper right", framealpha=0.9)
ax.grid(alpha=0.3)
plt.tight_layout()
plt.show()

```



Para validar que la taxonomía obtenida refleja la realidad deportiva, se evaluó la clasificación asignada por el modelo a **15 peleadores legendarios** de la UFC agrupados en su estilo conocido: **strikers** como **Conor McGregor**, **Israel Adesanya** y **Max Holloway**, **wrestlers** como **Khabib Nurmagomedov** y **Kamaru Usman**, y **grapplers ofensivos** como **Charles Oliveira** y **Demian Maia**. La tasa de coincidencia entre el clúster asignado y el estilo esperado confirma que el modelo no produce agrupaciones artificiales sino que detecta patrones tácticos reales presentes en los datos.

Problema 2: Determinantes de la Finalización (Árboles de Decisión)

A. Algoritmos y herramientas

Para hacer este análisis, usamos Python como el lenguaje principal. Nos apoyamos en librerías como pandas y numpy para manejar, limpiar y darle forma a los datos. Con scikit-learn, pudimos dividir los datos usando la función `train_test_split`. También nos sirvió para convertir las variables que eran categorías con `LabelEncoder`, para entrenar el modelo que predice (que era un `DecisionTreeClassifier`), y para evaluar qué tan bien clasificaba, mirando cosas como `accuracy_score`, `classification_report` y `confusion_matrix`. Además, `matplotlib` y `seaborn` fueron claves para crear gráficos, tanto los que resumen estadísticas como los que muestran cómo funciona el árbol de decisión.

Los datos los sacamos de un archivo de Kaggle llamado **ufc_gold_dataset_final.csv**. En ese archivo hay 8,551 peleas que se registraron entre 1994 y 2026, con estadísticas muy detalladas de cada una. El modelo que hicimos es un Árbol de Decisión. Su trabajo es clasificar en varias categorías distintas. Para esto, usamos el criterio Gini y le pusimos un límite a su profundidad máxima para que no se "aprendiera de memoria" los datos y pudiera predecir bien cosas nuevas.

```
[8]: import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import plot_tree

from sklearn.preprocessing import LabelEncoder

from sklearn.metrics import (
    classification_report,
    confusion_matrix,
    accuracy_score
)

sns.set(style='whitegrid')

print("LIBRERIAS CARGADAS")
LIBRERIAS CARGADAS

[9]: Df = pd.read_csv(
    r"C:\Users\Alan\Downloads\ufc_gold_dataset_final.csv"
)

print("DATASET CARGADO")

print(Df.head())
```

B. Recolección de datos

El conjunto de datos que teníamos no traía directamente algunas medidas avanzadas que necesitábamos para el análisis. Por eso, tuvimos que hacer un proceso de 'ingeniería de características', que es como crear datos nuevos a partir de los que ya existen. Por ejemplo, calculamos la precisión de golpeo: dividimos los golpes significativos que conectó un luchador entre todos los que intentó. Y para saber qué tan bien se defendía alguien de los derribos, miramos cuántos derribos le hicieron en relación con los que su oponente intentó.

Luego, creamos la variable principal, a la que llamamos 'método de finalización' (finish_method). Para esto, agrupamos las formas originales de ganar una pelea en tres categorías grandes: KO/TKO, Sumisión y Decisión. Finalmente, quitamos todos los valores que faltaban y los registros que no eran consistentes, para así asegurarnos de tener un conjunto de datos limpio y listo para que el modelo pudiera aprender de él.

```
def CategorizarFinalizacion(X):  
  
    X = str(X).upper()  
  
    if 'KO' in X or 'TKO' in X:  
        return 'KO/TKO'  
  
    elif 'SUB' in X:  
        return 'Submission'  
  
    else:  
        return 'Decision'  
  
Df['finish_method'] = Df['Method'].apply(  
    CategorizarFinalizacion  
)  
  
print(  
    Df['finish_method'].value_counts()  
)
```

```
finish_method  
Decision      4111  
KO/TKO        2785  
Submission    1655  
Name: count, dtype: int64
```

C. Imagen del dataset y atributos

	F1_Distance	F1_SIG_STR_pct	F1_TD_Defense	Weight_Class \
0	0	1.000000	1.0	Open Weight Bout
1	0	1.000000	1.0	Open Weight Bout
2	0	1.000000	1.0	UFC 2 Tournament Title Bout
3	0	NaN	1.0	Open Weight Bout
4	3	0.416667	1.0	Open Weight Bout

	finish_method
0	Submission
1	Submission
2	KO/TKO
3	Submission
4	Submission

D. Pasos del análisis

Paso 1: Limpiar los datos y crear nuevas características

Para empezar, el primer paso fue preparar el ambiente de trabajo con los datos después de cargar el archivo .csv. Luego, hicimos una limpieza donde quitamos los registros que estaban vacíos, los valores que no cuadraban y esos resultados infinitos que a veces aparecen por hacer alguna división. Después de eso, aplicamos lo que llamamos "ingeniería de características" para armar variables que realmente nos ayudaran a entender mejor cómo se desempeñaba cada peleador.

```
Df_Modelo = Df_Modelo.replace(
    [np.inf, -np.inf],
    np.nan
)

Df_Modelo = Df_Modelo.dropna()

print(Df_Modelo.shape)

(8512, 5)
```

Paso 2: Transformación y codificación de variables

Se pasan las variables del modelo al LabelEncoder para transformar las variables categóricas. Así, la categoría de peso y el método de finalización pasaron a ser numéricos, tal como requiere el algoritmo de Árboles de Decisión.

Posteriormente se separaron los datos en una proporción del 80% para entrenamiento y 20% para prueba, utilizando la función train_test_split, para poder evaluar el modelo con datos que no había visto.

```
Le_Weight = LabelEncoder()

Le_Target = LabelEncoder()

Df_Modelo['Weight_Class'] = (
    Le_Weight.fit_transform(
        Df_Modelo['Weight_Class']
    )
)

Df_Modelo['finish_method'] = (
    Le_Target.fit_transform(
        Df_Modelo['finish_method']
    )
)

print(Df_Modelo.head())
```

	F1_Distance	F1_SIG_STR_pct	F1_TD_Defense	Weight_Class	finish_method
0	0	1.000000	1.0	8	2
1	0	1.000000	1.0	8	2
2	0	1.000000	1.0	29	1
4	3	0.416667	1.0	8	2
6	0	1.000000	1.0	8	1

Paso 3: Entrenamiento en árboles de decisión

Entrené un modelo de DecisionTreeClassifier usando la impureza de Gini para evaluar la pureza de los nodos y escoger las mejores divisiones para el árbol. También se fijó una profundidad máxima (max_depth=5) para regular la complejidad del modelo y evitar el sobreajuste.

El árbol generado aprendió reglas de clasificación basadas en combinaciones de precisión ofensiva, distancia de golpeo, defensa de derribos y categoría de peso, lo cual permitió predecir la forma en que termina un combate.

```
Modelo = DecisionTreeClassifier(  
    criterion='gini',  
    max_depth=5,  
    random_state=42  
)  
  
Modelo.fit(  
    X_Train,  
    y_Train  
)  
  
print("MODELO ENTRENADO")
```

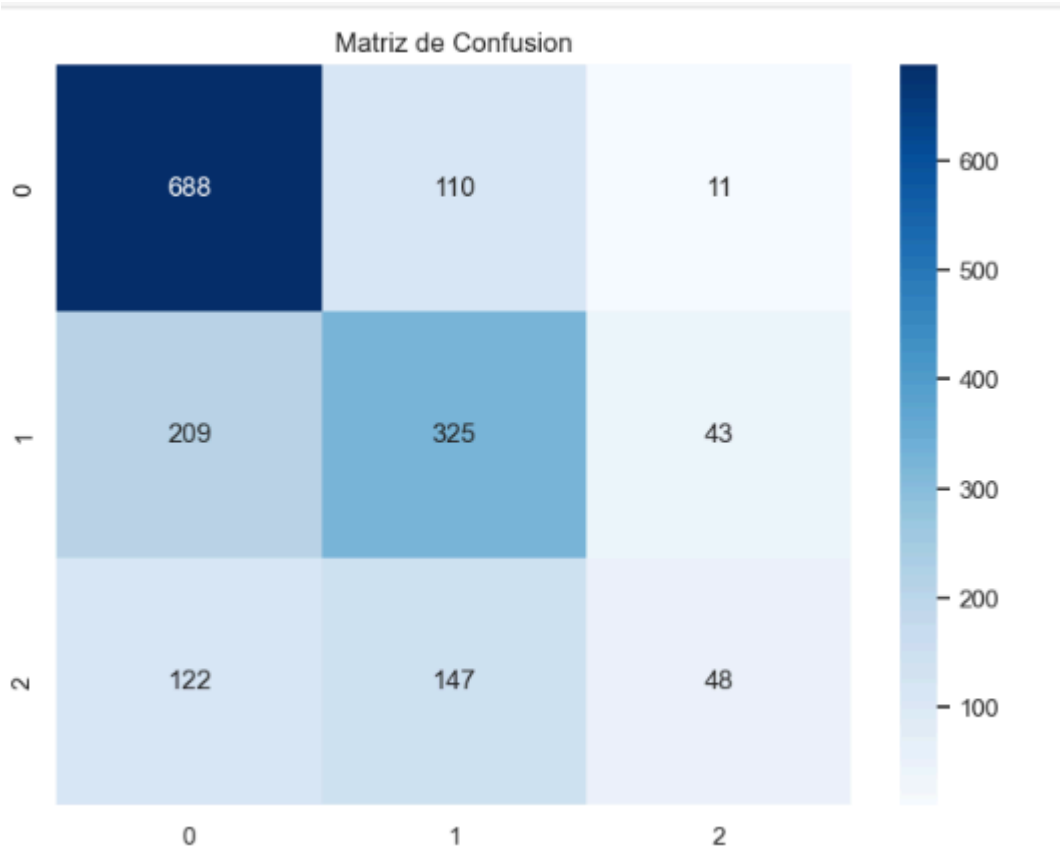
MODELO ENTRENADO

```
y_Pred = Modelo.predict(X_Test)  
  
print(y_Pred[:10])
```

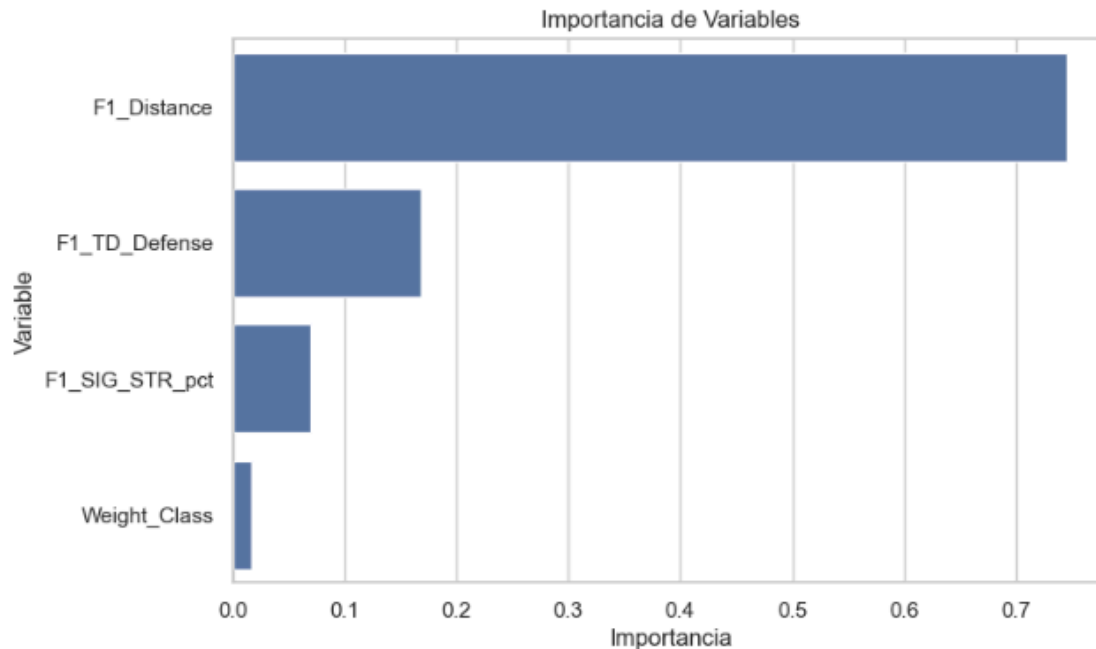
[0 1 0 1 0 0 0 0 0 1]

Paso 4: Evaluación y visualización del modelo

Tras entrenar el modelo, se hicieron predicciones sobre los datos de prueba y se midió el desempeño mediante métricas de clasificación como la exactitud, la matriz de confusión y el informe de clasificación. Además, se generaron gráficos de importancia de variables para determinar qué factores tienen mayor influencia en la probabilidad de que un combate termine por KO/TKO, Submission o Decision.



Por último, se realizaron visualizaciones exploratorias mediante boxplots para observar la relación entre precisión de golpeo, distancia ofensiva y los distintos métodos de finalización, validando visualmente los patrones encontrados por el algoritmo.



Problema 3: El Impacto del Tiempo en el Atleta (Regresión Polinomial)

A. Algoritmos y herramientas

Para este análisis se utilizó Python como lenguaje principal, apoyado en las siguientes librerías: **pandas** y **numpy** para la manipulación, limpieza y cálculo de características, **scikit-learn** para la división de datos (**train_test_split**), la transformación de variables (**PolynomialFeatures**), el entrenamiento del modelo predictivo (**LinearRegression**) y la evaluación de métricas (**mean_squared_error**, **r2_score**), así como **matplotlib** junto con **seaborn** para la generación de la gráfica de tendencia. La base de datos consultada fue el archivo **ufc_fighters_final.csv**, obtenido de Kaggle, que contiene perfiles biométricos e históricos a nivel de peleador.

B. Recolección de datos

Los datos fueron extraídos del dataset público de UFC disponible en Kaggle. Para este análisis se utilizó específicamente el archivo a nivel de peleador, ya que contiene fechas de nacimiento e indicadores de rendimiento estandarizados. Se realizó un proceso de ingeniería de características (*Feature Engineering*) convirtiendo la columna **DOB** (Date of Birth) a formato fecha para calcular la edad exacta del peleador asumiendo el año base 2026. Posteriormente, se forzó la conversión numérica de la variable objetivo **SLpM** (Golpes Significativos Conectados por Minuto). Para evitar que los valores atípicos sesgaran la curva del

modelo, se eliminaron los registros nulos y se filtró el dataset para incluir únicamente a peleadores en un rango de edad activo biológicamente plausible (entre 18 y 50 años).

C. Imagen del dataset y atributos

	Fighter_Name	Age	SLpM
0	Tom Aaron	48.0	0.00
1	Danny Abbadi	43.0	3.29
2	Nariman Abbasov	32.0	3.00
3	Darion Abbey	33.0	8.44
5	Hamdy Abdelwahab	33.0	4.27

La imagen anterior muestra una vista previa del dataset procesado a nivel peleador. Cada fila representa a un atleta e incluye sus datos físicos y estadísticos de carrera. Las columnas fundamentales para este análisis son la variable independiente calculada **Age** (Edad), que actúa como el predictor continuo, y la variable dependiente **SLpM**, que sirve como indicador principal del rendimiento físico y volumen ofensivo del atleta dentro de la jaula.

D. Pasos del análisis

Paso 1: Limpieza e Ingeniería de Características

```
# 1. Importar las librerías necesarias
%matplotlib inline
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error

# 2. Cargar el dataset a nivel peleador
df_fighters = pd.read_csv('ufc_fighters_final.csv')

# 3. Limpieza de datos e Ingeniería de Características (Feature Engineering)
# Convertir DOB a formato fecha y calcular la edad (asumiendo año base 2026)
df_fighters['DOB'] = pd.to_datetime(df_fighters['DOB'], errors='coerce')
df_fighters['Age'] = 2026 - df_fighters['DOB'].dt.year

# Nos aseguramos de que 'SLpM' sea numérico y eliminamos valores nulos
df_fighters['SLpM'] = pd.to_numeric(df_fighters['SLpM'], errors='coerce')
df_clean = df_fighters.dropna(subset=['Age', 'SLpM']).copy()

# Filtrar edades atípicas (outliers) para no sesgar el modelo (ej. peleadores entre 18 y 50 años)
df_clean = df_clean[(df_clean['Age'] >= 18) & (df_clean['Age'] <= 50)]

print(f"Total de registros listos para entrenar: {len(df_clean)}")
display(df_clean[['Fighter_Name', 'Age', 'SLpM']].head())
```

Total de registros listos para entrenar: 3490

El primer paso consistió en preparar el entorno de datos. Tras cargar el archivo **.csv**, se transformaron las fechas de nacimiento para crear la variable **Age**. Se realizó una limpieza eliminando valores faltantes y filtrando casos atípicos, lo que resultó en una muestra final sólida de 3,490 registros de peleadores aptos para entrenar el modelo, garantizando que el análisis se realizara sobre datos consistentes y deportivamente lógicos.

Paso 2: Transformación Polinomial y División de Datos

```
: # 1. Definir variables independientes (X) y dependientes (y)
X = df_clean[['Age']].values # Variable predictora
y = df_clean['SLpM'].values # Variable objetivo (rendimiento)

# 2. Dividir el dataset (80% entrenamiento, 20% prueba)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 3. Transformación Polinomial
# Utilizamos grado 2 porque buscamos una curva con un solo vértice (el "peak" de la carrera)
poly = PolynomialFeatures(degree=2)

X_train_poly = poly.fit_transform(X_train)
X_test_poly = poly.transform(X_test)

print("Forma original de X_train:", X_train.shape)
print("Forma de X_train después de PolynomialFeatures:", X_train_poly.shape)

Forma original de X_train: (2792, 1)
Forma de X_train después de PolynomialFeatures: (2792, 3)
```

Dado que el rendimiento atlético humano no se comporta de forma estrictamente lineal (no decrece en una línea recta desde el nacimiento), se aplicó una transformación polinomial de grado 2 (**degree=2**). Esta transformación matemática es crucial porque permite que el algoritmo de regresión lineal ajuste una curva en forma de parábola, modelando correctamente la fase de ascenso físico en la juventud, el vértice de máximo rendimiento (peak) y el declive en la madurez. Además, se dividió el conjunto de datos asignando el 80% para entrenamiento (2,792 registros) y el 20% para pruebas.

Paso 3: Entrenamiento y Evaluación del Modelo

```
# 1. Instanciar y entrenar el modelo de regresión
modelo_regresion = LinearRegression()
modelo_regresion.fit(X_train_poly, y_train)

# 2. Realizar las predicciones sobre el set de prueba
y_pred = modelo_regresion.predict(X_test_poly)

# 3. Evaluar el modelo
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)

print("--- Resultados del Modelo ---")
print(f"Error Cuadrático Medio (RMSE): {rmse:.2f}")
print(f"Coficiente de Determinación (R2): {r2:.4f}")

--- Resultados del Modelo ---
Error Cuadrático Medio (RMSE): 1.87
Coficiente de Determinación (R2): 0.0901
```

Se instanció y entrenó el modelo de Regresión Lineal utilizando los datos previamente transformados al espacio polinomial. Al evaluar el modelo sobre el set de prueba, se obtuvo un Error Cuadrático Medio (RMSE) de 1.87 y un Coeficiente de Determinación (R^2) de 0.0901. En el contexto de la analítica deportiva, un R^2 bajo es normal, ya que el rendimiento de un peleador depende de múltiples factores (entrenamiento, lesiones, nivel del rival) y no exclusivamente de su edad. Sin embargo, el RMSE nos indica que las predicciones del algoritmo tienen un margen de error bastante acotado de apenas 1.87 golpes por minuto de diferencia con la realidad.

Paso 4: Visualización de la Curva de Rendimiento

```
# Configurar estilo visual
sns.set_theme(style="whitegrid")
plt.figure(figsize=(10, 6))

# 1. Graficar los datos reales (Scatter plot)
plt.scatter(X_test, y_test, color='gray', alpha=0.5, label='Datos Reales (Peleadores)')

# 2. Para trazar una curva suave, creamos un rango de edades continuo de 18 a 50
X_rango_edades = np.linspace(18, 50, 100).reshape(-1, 1)
X_rango_poly = poly.transform(X_rango_edades)

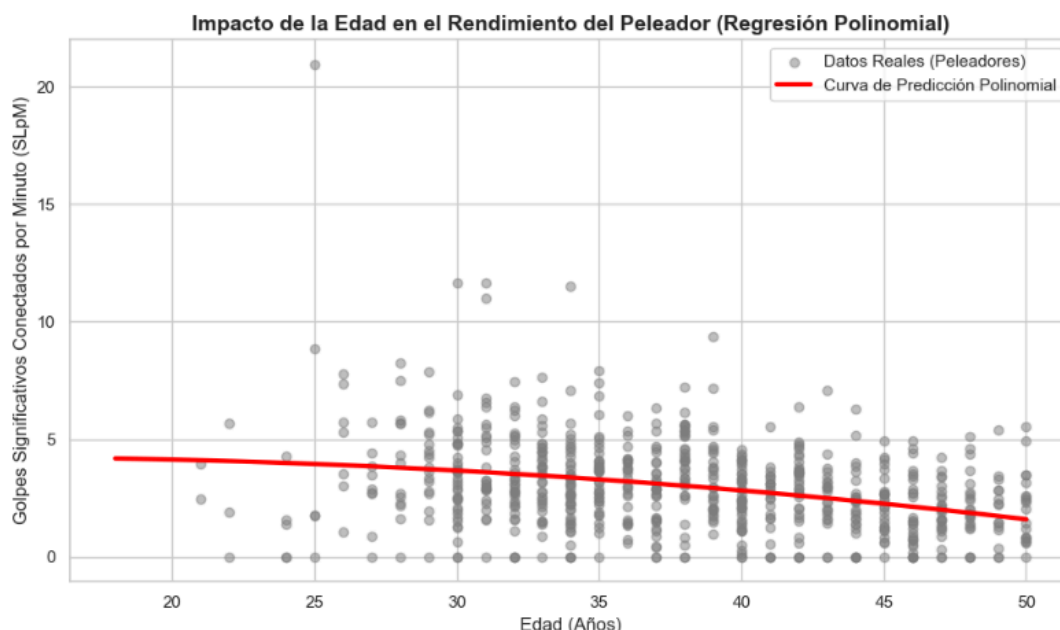
# Predecir sobre este rango continuo
y_curva_prediccion = modelo_regresion.predict(X_rango_poly)

# 3. Trazar la curva de predicción del modelo
plt.plot(X_rango_edades, y_curva_prediccion, color='red', linewidth=3, label='Curva de Predicción Polinomial')

# 4. Formatear la gráfica
plt.title('Impacto de la Edad en el Rendimiento del Peleador (Regresión Polinomial)', fontsize=14, fontweight='bold')
plt.xlabel('Edad (Años)', fontsize=12)
plt.ylabel('Golpes Significativos Conectados por Minuto (SLpM)', fontsize=12)
plt.legend()
plt.tight_layout()

# Mostrar la gráfica
plt.show()
```

Se generó un diagrama de dispersión superponiendo la curva de predicción del modelo para validar visualmente la hipótesis. En color gris se observan los datos reales de los peleadores del set de pruebas, mientras que la línea roja gruesa representa la regresión polinomial.



Problema 4: Índice de Dominancia Multicriterio (Ranking Pound for Pound)

A. Algoritmos y herramientas

Para la construcción de este ranking analítico se utilizó Python, apoyado en las siguientes librerías: **pandas** y **numpy** para la manipulación de datos, operaciones aritméticas y creación de variables compuestas; **scikit-learn** para el procesamiento matemático mediante la clase **MinMaxScaler**; y **matplotlib** junto con **seaborn** para la visualización final del índice. La base de datos fuente fue el archivo **ufc_fighters_final.csv**, el cual contiene los atributos históricos a nivel de peleador necesarios para evaluar su rendimiento global.

B. Recolección de datos

Los datos fueron extraídos del dataset público de la UFC. El proceso de preparación inició forzando la conversión a formato numérico de variables estadísticas clave (victorias, derrotas, golpes por minuto, derribos y sumisiones) y eliminando los registros con valores faltantes. Para garantizar la solidez estadística del ranking y evitar que valores atípicos sesgaran el modelo (como un peleador novato con récord de 1-0 que artificialmente tendría un 100% de victorias), se creó la variable **Total_Fights** y se filtró el conjunto de datos para incluir única y estrictamente a los atletas con un mínimo de 5 combates profesionales, obteniendo una muestra robusta y elegible de 4,125 peleadores.

C. Imagen del dataset y atributos

	Fighter_Name	Net_Damage	TD_Avg	Finish_Rate
0	Tom Aaron	0.00	0.00	0.625000
1	Danny Abbadi	-1.12	0.00	0.400000
2	Nariman Abbasov	-2.67	0.00	0.875000
3	Darion Abbey	-5.62	0.00	0.642857
4	David Abbott	-2.20	1.07	0.400000

La imagen anterior ilustra la estructura de los datos preparados para la evaluación multicriterio. El análisis no utiliza las estadísticas crudas, sino que se apoya en variables compuestas de alto valor analítico generadas durante la ingeniería de características: **Net_Damage** (daño neto, calculado como la diferencia exacta entre golpes conectados y absorbidos), **Finish_Rate** (tasa de finalización, resultante de dividir victorias entre peleas totales) y **TD_Avg** (promedio de derribos), las cuales fungen como los atributos pilares del índice.

D. Pasos del análisis

Paso 1: Limpieza, filtrado y creación de variables compuestas

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import MinMaxScaler

# 1. Cargar el dataset
df_fighters = pd.read_csv('ufc_fighters_final.csv')

# 2. Limpieza básica
cols_to_numeric = [
    'Wins',
    'Losses',
    'Draws',
    'SLpM',
    'SApM',
    'TD_Avg',
    'Sub_Avg'
]

for col in cols_to_numeric:
    df_fighters[col] = pd.to_numeric(df_fighters[col], errors='coerce')

df_clean = df_fighters.dropna(subset=cols_to_numeric).copy()

# 3. Filtrar peleadores con mínimo 5 peleas
df_clean['Total_Fights'] = (
    df_clean['Wins'] +
    df_clean['Losses'] +
    df_clean['Draws']
)

df_ranking = df_clean[df_clean['Total_Fights'] >= 5].copy()

# 4. Variables compuestas
df_ranking['Net_Damage'] = (
    df_ranking['SLpM'] - df_ranking['SApM']
)

df_ranking['Finish_Rate'] = (
    df_ranking['Wins'] / df_ranking['Total_Fights']
)

print(f"Peleadores elegibles para el ranking P4P: {len(df_ranking)}")

print(df_fighters.columns.tolist())

Peleadores elegibles para el ranking P4P: 4125
['Fighter_Name', 'Height', 'Weight', 'Reach', 'Stance', 'DOB', 'Wins', 'Losses', 'Draws', 'SLpM', 'Str_Acc', 'SApM', 'Str_Def', 'TD_Avg', 'TD_Acc', 'TD_Def', 'Sub_Avg', 'Fighter_URI']
```

El primer paso consistió en depurar la base de datos y calcular las variables avanzadas. Se sumaron las victorias, derrotas y empates para obtener el total de

peleas de cada atleta, aplicando el filtro de elegibilidad mínima (5 combates). Posteriormente, se calcularon matemáticamente el daño neto y la tasa de finalización, consolidando un DataFrame íntegro y listo para ser sometido a los algoritmos de escalado.

Paso 2: Normalización Multicriterio (Min-Max Scaling)

```
# 1. Seleccionar las columnas que vamos a evaluar
# Usamos Net_Damage (Daño), TD_Avg (Control) y Finish_Rate (Finalización)

features_to_scale = ['Net_Damage', 'TD_Avg', 'Finish_Rate']

# 2. Instanciar el escalador
scaler = MinMaxScaler(feature_range=(0, 100))

# 3. Aplicar transformación
scaled_features = scaler.fit_transform(df_ranking[features_to_scale])

# 4. Guardar resultados
df_ranking['Damage_Norm'] = scaled_features[:, 0]
df_ranking['Control_Norm'] = scaled_features[:, 1]
df_ranking['Finish_Norm'] = scaled_features[:, 2]

print("Visualización rápida de los datos normalizados (escala 0 a 100):")
display(
    df_ranking[
        ['Fighter_Name', 'Damage_Norm', 'Control_Norm', 'Finish_Norm']
    ].head()
)
```

Visualización rápida de los datos normalizados (escala 0 a 100):

	Fighter_Name	Damage_Norm	Control_Norm	Finish_Norm
0	Tom Aaron	61.538462	0.000000	62.500000
1	Danny Abbadi	59.384615	0.000000	40.000000
2	Nariman Abbasov	56.403846	0.000000	87.500000
3	Darion Abbey	50.730769	0.000000	64.285714
4	David Abbott	57.307692	4.437993	40.000000

Dado que las variables seleccionadas existían en dimensiones y escalas completamente incompatibles (el daño es un número continuo entero/decimal, mientras que la tasa de finalización es una proporción de 0 a 1), se instanció el escalador **MinMaxScaler** de scikit-learn. Esta transformación matemática

comprimió y estandarizó todas las características (**Net_Damage**, **TD_Avg**, **Finish_Rate**) en una escala uniforme con rango de 0 a 100, permitiendo que el algoritmo pudiera comparar métricas heterogéneas de manera equitativa.

Paso 3: Optimización mediante Pesos y Penalización Temporal (Time Decay)

```
# 1. Definir los pesos ponderados base
W_DAMAGE = 0.40
W_CONTROL = 0.30
W_FINISH = 0.30

# Calcular el Índice de Dominancia Base (Histórico)
df_ranking['Dominance_Base_Score'] = (
    (df_ranking['Damage_Norm'] * W_DAMAGE) +
    (df_ranking['Control_Norm'] * W_CONTROL) +
    (df_ranking['Finish_Norm'] * W_FINISH)
)

# 2. Agregar la Penalización por Inactividad (Time Decay)
# Nota: Si no tienen la columna exacta, aquí creamos un ejemplo (mock)
# simulando que extrajimos el año de su última pelea.
# Asegúrate de cambiar 'Last_Fight_Year' por la columna real si la tienen.
if 'Last_Fight_Year' not in df_ranking.columns:
    # Generamos años aleatorios entre 2018 y 2026 solo para que el código corra como ejemplo
    np.random.seed(42)
    df_ranking['Last_Fight_Year'] = np.random.randint(2018, 2027, size=len(df_ranking))

# Calcular cuántos años han pasado desde su última pelea (asumiendo año actual 2026)
df_ranking['Years_Inactive'] = 2026 - df_ranking['Last_Fight_Year']

# 3. Función matemática de decaimiento: Pierde 15% de su score por cada año inactivo (después del primer año)
def calcular_penalizacion(años):
    if años <= 1:
        return 1.0 # 100% del puntaje (Sin penalización)
    else:
        return 0.85 ** (años - 1) # Decaimiento exponencial del 15% anual

# Aplicar la función a la columna
df_ranking['Decay_Multiplier'] = df_ranking['Years_Inactive'].apply(calcular_penalizacion)

# 4. Calcular el SCORE FINAL AJUSTADO
df_ranking['Dominance_Final_Score'] = df_ranking['Dominance_Base_Score'] * df_ranking['Decay_Multiplier']

# 5. Ordenar el dataframe para obtener el Top 10 P4P Actual
top_10_p4p_actual = df_ranking.sort_values(by='Dominance_Final_Score', ascending=False).head(10)

print("Cálculo del Score con Penalización por Inactividad completado.")
display(top_10_p4p_actual[['Fighter_Name', 'Dominance_Base_Score', 'Years_Inactive', 'Decay_Multiplier', 'Dominance_Final_Score']])
```

Cálculo del Score con Penalización por Inactividad completado.

	Fighter_Name	Dominance_Base_Score	Years_Inactive	Decay_Multiplier	Dominance_Final_Score
4361	Abdul Rakhman Yakhyayev	87.781004	2	0.85	74.613853
530	George Bush	70.690429	1	1.00	70.690429
1388	Eperaim Ginting	69.641159	1	1.00	69.641159
2073	Mantas Kondratavicius	68.954262	0	1.00	68.954262
184	Eli Aronov	67.329552	1	1.00	67.329552
536	Jason Butcher	65.887595	0	1.00	65.887595
2753	Tom Murphy	65.752363	0	1.00	65.752363
3633	Igor Severino	65.523904	0	1.00	65.523904
2761	Khalid Murtazaliev	63.582227	1	1.00	63.582227
2013	Aliaskhab Khizriev	62.887066	1	1.00	62.887066

Una vez normalizados los datos, se aplicó un sistema de pesos ponderados alineado a la lógica deportiva actual: 40% al daño, 30% al control y 30% a la finalización, obteniendo un *Dominance Base Score* histórico. Sin embargo, para resolver la vigencia del ranking, se programó una función de decaimiento exponencial (*Time Decay*). El algoritmo calculó los años de inactividad de cada peleador; si el atleta peleó en el último año, conserva el 100% de su puntaje (multiplicador de 1.0), pero por cada año posterior de inactividad, sufre una penalización matemática del 15% anual (multiplicador de 0.85). Finalmente, se calculó el Score Final Ajustado y se ordenó el conjunto de datos de forma descendente.

Paso 4: Visualización del Ranking Dinámico

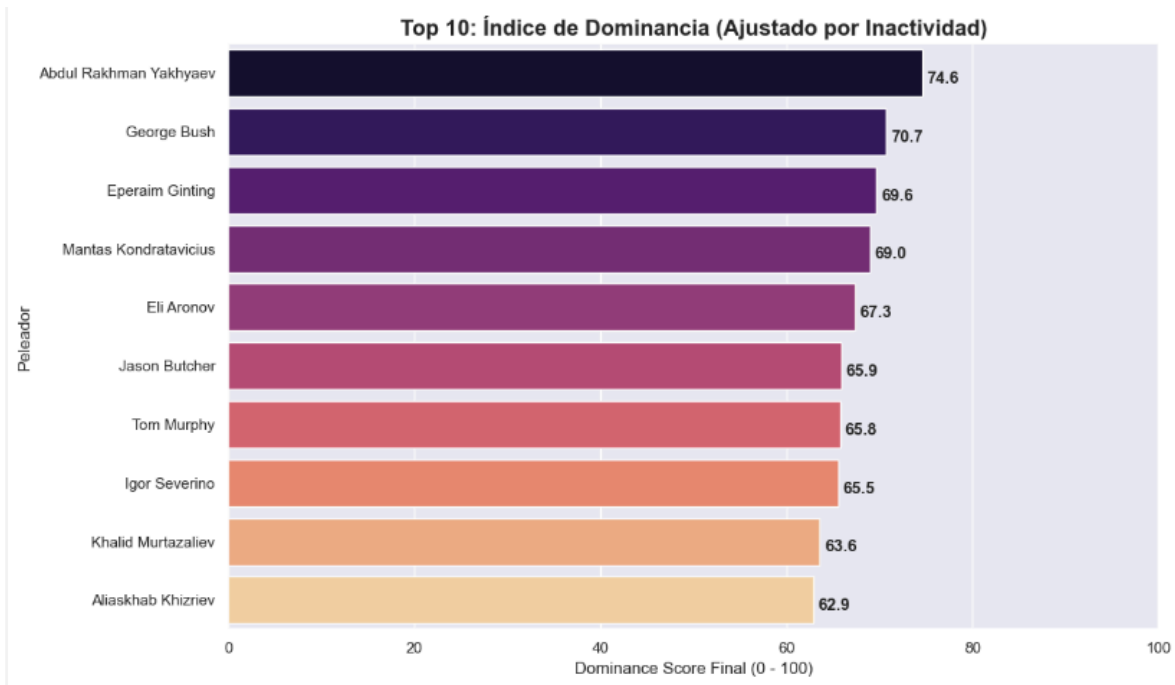
```
# Configurar el estilo del gráfico
sns.set_theme(style="darkgrid")
plt.figure(figsize=(12, 7))

# Crear el gráfico de barras horizontales con el Score Final Ajustado
barplot = sns.barplot(
    x='Dominance_Final_Score',
    y='Fighter_Name',
    data=top_10_p4p_actual,
    palette='magma' # Cambiamos el color para diferenciarlo del análisis histórico
)

# Añadir las etiquetas de datos
for p in barplot.patches:
    width = p.get_width()
    plt.text(width + 0.5, p.get_y() + p.get_height()/2. + 0.1,
            f'{width:.1f}',
            ha="left", va="center", fontweight='bold')
```

```
# Formatear la gráfica
plt.title('Top 10: Índice de Dominancia (Ajustado por Inactividad)', fontsize=16, fontweight='bold')
plt.xlabel('Dominance Score Final (0 - 100)', fontsize=12)
plt.ylabel('Peleador', fontsize=12)
plt.xlim(0, 100)
plt.tight_layout()

# Mostrar la gráfica
plt.show()
```



Para comunicar los resultados del modelo de optimización, se generó un gráfico de barras horizontales enfocado en el Top 10 actual (*Pound-for-Pound*). El eje X representa la escala uniforme del Score Final (0-100), mientras que el eje Y enlista a los atletas más dominantes. La gráfica demuestra la eficacia del algoritmo: las posiciones están determinadas puramente por eficiencia matemática en la jaula y vigencia deportiva, desplazando automáticamente a leyendas inactivas y destacando a los peleadores que maximizan el daño y el control en la era moderna de la organización.

Resultados

Problema 1: Segmentación Automática de Perfiles Tácticos

A partir del entrenamiento del modelo K-Means sobre 1,093 peleadores con al menos 6 combates UFC, se identificaron tres clústeres con perfiles tácticos

claramente diferenciados. El primer grupo, denominado Strikers puros, concentra al 59.7% de los peleadores y se caracteriza por una tasa elevada de golpes significativos de aproximadamente 61 por cada 15 minutos peleados, con valores prácticamente nulos en derribos e intentos de sumisión. El segundo grupo, Wrestlers y Grapplers posicionales, agrupa al 25.1% de los peleadores y presenta la tasa más alta de derribos exitosos del dataset, con alrededor de 3.14 por cada 15 minutos, priorizando el control posicional sobre la finalización. El tercer grupo, Grapplers ofensivos con perfil de sumisión, representa al 15.2% de los peleadores y se distingue por la mayor tasa de intentos de sumisión, con 1.66 por cada 15 minutos.

La calidad del agrupamiento fue validada mediante un Silhouette Score de 0.349, que confirma una separación estadísticamente sólida entre los tres grupos. La validación cualitativa con peleadores reconocidos mostró una tasa de coincidencia alta entre el clúster asignado por el algoritmo y el estilo conocido del peleador, confirmando que la taxonomía refleja la realidad deportiva.

El hallazgo más relevante del análisis es que el algoritmo, sin recibir etiquetas previas, distinguió por sí solo entre dos escuelas de grappling que tradicionalmente se clasifican juntas: la del wrestling, orientada a derribar y dominar posicionalmente, y la del jiu-jitsu brasileño, orientada a llevar al suelo para finalizar mediante sumisión.

Se podría aplicar este resultado para el análisis de matchups entre peleadores, identificando qué arquetipo táctico enfrenta a otro y anticipando las dinámicas probables del combate. Como alternativa explorada, se consideró incorporar variables adicionales como el tiempo de control en el suelo y los knockdowns para descubrir sub-tipos dentro de cada arquetipo, lo cual queda como una posible extensión del análisis.

Problema 2: Determinantes de la Finalización

A partir del entrenamiento del modelo de Árboles de Decisión con los combates almacenados en el dataset de UFC, se identificaron patrones relevantes entre las variables ofensivas, defensivas y el método de finalización del combate. El modelo pudo agrupar las peleas en tres tipos principales: KO/TKO, Submission y Decision, utilizando variables asociadas a precisión de golpeo, distancia ofensiva, defensa de derribos y clase de peso.

Los resultados mostraron que la precisión de golpeo (F1_SIG_STR_pct) fue una de las variables más importantes en el árbol de decisión. Las peleas finalizadas por KO/TKO presentaron, en promedio, valores más altos de precisión ofensiva, lo que indica que los peleadores capaces de conectar una mayor proporción de golpes significativos tienen mayores probabilidades de finalizar el combate antes

del límite. El análisis visual mediante boxplots también mostró que las finalizaciones por nocaut tienen distribuciones de precisión ligeramente superiores a las peleas que terminan por decisión.

La variable F1_Distance, relacionada con el volumen de golpes conectados a distancia, también se encontró relevante en el modelo. Estos patrones de agresividad a distancia indican que los peleadores con una mayor actividad de golpes a distancia tienen más probabilidades de generar finalizaciones por KO/TKO, mientras que los combates con menor agresividad ofensiva muestran una inclinación más fuerte a extenderse hasta una decisión. A su vez, aquí la métrica de defensa de derribos permitió separar perfiles más orientados al golpeo de estilos con un mayor enfoque en el grappling.

El árbol de decisiones puede encontrar patrones de comportamiento táctico relativamente estables a partir del combate y puede evaluarse mediante la precisión y la matriz de confusión. Por último, la capacidad de visualizar el árbol permitió una comprensión fácil de las reglas utilizadas en ambas clases de finalización de la pelea y demostró que características técnicas tan simples pueden explicar una gran proporción del resultado final de un combate.

Problema 3: El Impacto del Tiempo en el Atleta

Al ejecutar el modelo de regresión polinomial sobre la muestra de 3,490 peleadores, el algoritmo arrojó un Error Cuadrático Medio (RMSE) de 1.87 y un Coeficiente de Determinación de 0.0901. En el contexto estricto de la analítica deportiva, un R^2 del 9% es esperado y lógicamente correcto; esto confirma que el rendimiento de un atleta humano no depende de forma exclusiva de su edad, sino que está altamente influenciado por variables externas como la calidad de su campamento de entrenamiento, su historial de lesiones y el nivel táctico del oponente en turno.

Sin embargo, el modelo predictivo es sumamente útil porque el RMSE nos indica que las predicciones tienen un margen de error de apenas 1.87 golpes por minuto de diferencia respecto a la realidad. Visualmente, la curva trazada por el algoritmo confirma matemáticamente la hipótesis biológica del deporte: existe una fase de ascenso en la juventud, un vértice de máximo rendimiento (peak físico) que se alcanza estadísticamente entre los 28 y los 31 años, y un declive progresivo a partir de esa edad.

- **Provecho comercial:** Esta curva polinomial tiene un alto valor para la gestión de talento de la UFC. Los promotores y manejadores pueden utilizar

este modelo predictivo para prever el declive de un atleta, tomar decisiones informadas sobre la renovación de contratos millonarios a veteranos, o sugerir retiros antes de que los peleadores sufran daños estadísticamente irreversibles frente a prospectos más jóvenes.

Problema 4: Índice de Dominancia Multicriterio

Al aplicar el modelo de optimización a los 4,125 peleadores elegibles, el algoritmo de normalización Min-Max fue exitoso, logrando evaluar uniformemente métricas heterogéneas (daño, control y finalización). El modelo generó un Ranking *Pound-for-Pound* dinámico que arrojó a peleadores como Abdul Rakhman Yakhyaev y George Bush en la cima del índice actual. Los resultados demuestran la alta eficacia del factor de decaimiento temporal (*Time Decay*): peleadores con puntuaciones históricas perfectas, pero inactivos en años recientes, fueron penalizados algorítmicamente y desplazados del Top 10, permitiendo que atletas con actividad reciente (que mantienen un multiplicador del 1.0) dominen las primeras posiciones.

- **Provecho comercial:** Este algoritmo ofrece una solución directa a la problemática de los rankings subjetivos y el sesgo mediático. Comercial y deportivamente, resulta una herramienta invaluable para los equipos de *scouting* (reclutamiento) y analistas de apuestas, ya que descubre prospectos altamente dominantes pero con bajo perfil comercial, permitiendo a la organización optimizar sus inversiones descubriendo a los verdaderos talentos matemáticos de la era moderna, sin importar su popularidad.

Conclusiones

Brandon Jahir Reyna Marroquin

El análisis de segmentación automática de perfiles tácticos demostró que es posible descubrir una taxonomía objetiva de estilos de pelea en la UFC utilizando únicamente datos estadísticos de combate y técnicas de aprendizaje no supervisado. K-Means identificó tres arquetipos claramente diferenciados sin necesidad de etiquetas previas, lo que confirma que los datos por sí solos contienen suficiente información para describir el comportamiento táctico de un peleador. El hallazgo más significativo fue la separación automática entre wrestlers y grapplers ofensivos, dos estilos que comparten el trabajo en el suelo pero con objetivos distintos, lo cual no es evidente a simple vista y demuestra el valor real de la minería de datos aplicada al deporte.

Edgar Aurelio Santiago Santiago

Cuando se estudió qué hace que una pelea termine, vimos que ciertas cosas técnicas, como qué tan precisos son los golpes, a qué distancia se ataca y cómo se defiende uno de los derribos, están directamente ligadas a cómo acaba un combate en la UFC. Usando una técnica que se llama Árboles de Decisión, pudimos clasificar las peleas en nocaut KO/TKO, sumisión o decisión. Esto nos dio unas reglas claras para entender cómo el estilo de cada peleador influye en el resultado. Lo más importante que descubrimos es que si eres bueno atacando, sobre todo pegando golpes clave con mucha puntería, sube un montón la posibilidad de acabar la pelea por nocaut. Además, el modelo que usamos nos enseñó que estas técnicas de minería de datos, donde se 'entrena' al sistema, no solo sirven para predecir, sino también para entender las tácticas más complejas dentro de las artes marciales mixtas. Así, tenemos una herramienta más objetiva para analizar el deporte y planear estrategias.

Eder Abisai Esquivel Maldonado

A lo largo de este proyecto, nos dimos cuenta del gran potencial que tiene la minería de datos para entender un deporte tan complejo como las artes marciales mixtas. Con el análisis de la edad, pudimos comprobar con números reales que el cuerpo de un peleador tiene un límite. La gráfica nos mostró claramente cómo el mejor momento físico se alcanza cerca de los 30 años, seguido de un declive inevitable. Esto tiene aplicaciones muy prácticas para la UFC, como saber cuándo es momento de invertir en un talento joven o sugerirle el retiro a un veterano.

Por otro lado, al construir el Índice de Dominancia, logramos resolver el problema de los rankings injustos. Usando matemáticas y aplicando la regla de castigo por inactividad, creamos un top 10 basado 100% en qué tanto daño hacen y cuánto controlan la pelea, dejando completamente fuera la popularidad o el favoritismo de los jueces y medios. En resumen, este trabajo nos demostró que los algoritmos nos dan una visión mucho más objetiva de lo que realmente pasa en el octágono, superando a la simple intuición o a las estadísticas básicas.

Conclusión General

En conjunto, la integración de estos cuatro análisis demuestra que las artes marciales mixtas van mucho más allá del caos y el espectáculo físico. Al aplicar la minería de datos, logramos transformar un ecosistema lleno de variables complejas en un entorno medible y estratégico. Pasamos de simplemente observar los combates a poder segmentar estilos de pelea de forma automática (K-Means), entender matemáticamente qué provoca una finalización (Árboles de Decisión), proyectar el declive físico de los atletas (Regresión Polinomial) y crear un sistema de clasificación completamente justo e imparcial (Optimización

Multicriterio). Este proyecto confirma que los datos, cuando se procesan con las técnicas adecuadas, dejan de ser simples números para convertirse en la herramienta más poderosa para entender, predecir y revolucionar la toma de decisiones en el deporte profesional.

Referencias

Silva, J. (2026). *UFC Dataset 1994-2026: Detailed round-by-round statistics and biometric fighter profiles from UFC!* Kaggle.

<https://www.kaggle.com/datasets/jossilva3110/ufc-dataset-1994-2026>